



SAPIENZA
UNIVERSITÀ DI ROMA

Basi di Dati 1

UNIVERSITÀ DEGLI STUDI DI ROMA "LA
SAPIENZA"

FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
PROF.SSA MARIA DE MARSICO

31 gennaio 2024



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Indice

Prefazione	1
1 Introduzione al corso	5
1.1 Introduzione alle basi di dati	5
1.2 Storia delle basi di dati	6
2 Modello relazionale	7
2.1 Vincoli di integrità	8
2.2 Chiavi	9
2.3 Dipendenze funzionali	9
3 Algebra relazionale	11
3.1 Proiezione e Selezione	11
3.2 Unione, Differenza e Intersezione	12
3.3 Prodotto Cartesiano e Ridenominazione	14
3.4 Join Naturale e Theta Join	16
3.5 Quantificazione universale	18
4 Progettazione di una base di dati	21
4.1 Problemi e vincoli	21
4.2 Dipendenze funzionali	22
4.2.1 Chiusura di F	23
4.3 Assiomi di Armstrong	24
4.3.1 Chiusura di un insieme di attributi	25
4.3.2 $F^+ = F^A$	25
5 Terza forma normale (3NF)	29
5.1 Dipendenze parziali e transitive	30
5.2 Forma normale di Boyce-Codd	31
5.3 Chiusura di un insieme di attributi	31
5.3.1 Identificare le chiavi di uno schema	33
5.4 Decomposizione di uno schema	35
5.4.1 Preservare le dipendenze	35
5.4.2 Join senza perdita	39
5.4.3 Copertura minimale	42
5.4.4 Algoritmo di decomposizione	45
6 Organizzazione fisica dei dati	47
6.1 Record	47
6.2 Blocchi	47
6.3 File	48
6.3.1 File heap	48
6.3.2 File hash	49
6.3.3 File con indice (ISAM)	52
6.3.4 File B-tree	56

7	Controllo della concorrenza	59
7.1	Introduzione alla concorrenza	59
7.2	Item	61
7.3	Locking	61
7.3.1	Lock binario	61
7.4	Deadlock e livelock	63
7.5	Locking a due fasi stretto	64
7.6	Timestamp	65

Capitolo 1

Introduzione al corso

Il primo modulo del corso di Basi di Dati tratterà i seguenti argomenti: algebra relazionale, progettazione di una base di dati, terza forma normale (3NF), organizzazione fisica dei dati e controllo della concorrenza.

1.1 Introduzione alle basi di dati

Un sistema informativo è il componente di un'organizzazione che viene utilizzato per gestire (acquisire, processare, memorizzare, comunicare) le informazioni di interesse. Normalmente opera a supporto delle altre componenti dell'organizzazione ed è indipendente dalla sua computerizzazione.

Prima dell'avvento delle basi di dati, ogni applicazione era scritta in un linguaggio orientato alla gestione di file (Cobol, PL/1) e aveva il suo file privato ad organizzazione sequenziale. Ovviamente, un sistema di questo tipo porta con sé molti svantaggi tra cui: ridondanza, se due applicazioni usavano gli stessi dati questi venivano replicati; inconsistenza, l'aggiornamento di un dato poteva riguardare una sola copia del dato; dipendenza dei dati, ogni applicazione organizzava i dati tenendo conto dell'uso che doveva farne.

Una **Base di Dati** (Database – DB) è un insieme di file mutuamente connessi. Questi insiemi sono organizzati in diverse strutture dati che ne facilitano la creazione, l'accesso e l'aggiornamento ed ottimizzano la gestione delle risorse fisiche. Quindi l'obiettivo di una base di dati è facilitare l'elaborazione dei dati sulla base delle loro relazioni.

I **Sistemi di Gestione di Basi di Dati** (Database Management System – DBMS) sono strumenti software per la gestione di grandi masse (strutturate, processabili, condivise) di dati residenti su memoria secondaria.

Quando si registrano dei dati in formato elettronico si possono avere dei dati strutturati oppure dei dati non strutturati. I *dati strutturati* possono essere rappresentati dalle strutture dati gestite da un programma (stringhe, numeri...), mentre quelli *non strutturati* sono quei testi scritti in un linguaggio naturale. L'utilità nell'utilizzo dei dati strutturati proviene dal fatto che si può accedere singolarmente agli elementi della struttura tramite "interrogazioni" per recuperare le informazioni necessarie, oppure si possono effettuare dei calcoli sui dati stessi.

Una base di dati è una risorsa integrata condivisa da diverse componenti e, l'integrazione e la condivisione, permettono di ridurre le ridondanze e le inconsistenze dei dati. Da notare che la condivisione dei dati non è mai completa (controllo della privacy e regolamentazione degli accessi) e comporta la necessità di gestire accessi contemporanei agli stessi dati (controllo della concorrenza).

Nelle basi di dati possiamo distinguere i modelli logici da quelli concettuali. I *modelli logici* ci permettono di astrarre dall'organizzazione fisica, in fase di progettazione, invece, occorre avere dei *modelli concettuali* che ci permettono di rappresentare le entità del mondo reale e le loro relazioni.

Ogni database contiene tre livelli di astrazione (vedi Figura 1.1): uno *schema interno*, ovvero il modo in cui vengono memorizzate le tabelle e gli indici nei file, è chiamato anche schema fisico che rappresenta la distribuzione dei dati su disco; lo *schema logico* è la descrizione dell'intera base di dati nel modello principale del DBMS; gli *schema esterni* sono la descrizione di una porzione della base di

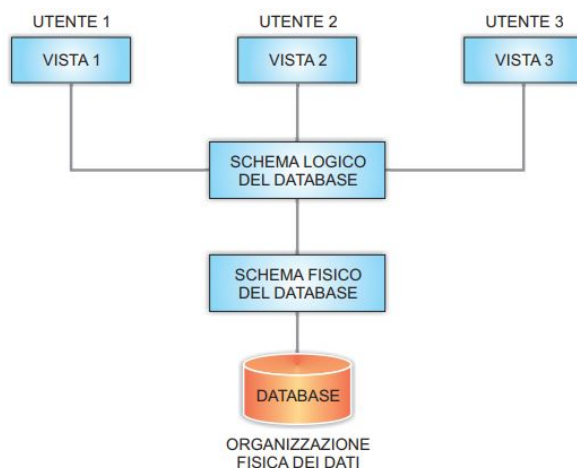


Figura 1.1: Livelli di astrazione dei sistemi per database

dati attraverso viste parziali che riflettono esigenze e privilegi di accesso di particolari tipologie di utenti. Gli accessi alla base di dati avvengono solamente attraverso lo schema esterno.

Per le basi di dati esistono un *data definition language* (DDL), un linguaggio per la definizione di schemi (logici, esterni, fisici) e altre operazioni generali, e un *data manipulation language* (DML), un linguaggio per l'interrogazione e l'aggiornamento di istanze di basi di dati. L'SQL (Structured Query Language) è un linguaggio standardizzato per database basati sul modello relazionale (RDBMS). In SQL i due tipi di funzionalità sono integrate in un unico linguaggio di comandi.

1.2 Storia delle basi di dati

I primi database gerarchici, progettati dalla IBM, furono usati nel progetto Apollo del 1964, mentre il primo database in commercio, DL/1, arriva qualche anno più tardi. Vengono chiamati database gerarchici perché prevedono delle gerarchie tra i dati con dei puntamenti tramite indirizzi fisici, la struttura di un database gerarchico è quella di un albero dove ogni nodo ha solo un genitore. Con l'avvento dei database a rete è possibile rappresentare un maggior numero di relazioni, con la rappresentazione dei dati tramite una collezione di record di tipo omogeneo, e con l'implementazione dei puntatori come relazioni binarie. Il modello di un database reticolare è rappresentato da una struttura a grafo dove i nodi sono i record e gli archi sono le relazioni. Nel 1970, con il modello di Codd, si introduce il modello relazionale dove i dati e le relazioni sono rappresentati come valori univoci di un record. Verso la metà degli anni '80 arrivarono i primi database orientati agli oggetti.

Capitolo 2

Modello relazionale

Il *modello relazionale* è basato sulla nozione di matematica di relazione. Le relazioni si traducono in maniera naturale in tabelle, dove dati e relazioni diverse sono rappresentati come valori. Nel modello relazionale, ogni oggetto è rappresentato con un record (o *tupla*) e le informazioni di interesse sono i campi del record. Una tabella (o *istanza*) è un insieme di record di tipo omogeneo e uno *schema* è un insieme di attributi che descrivono gli oggetti di cui vogliamo raccogliere le informazioni. Le n-uple rappresentano ogni singolo oggetto dove viene assegnato un valore a ognuno degli attributi che sono stati definiti per descriverlo.

Esempio 2.0.1 – Database relazionale

Studenti			
Matricola	Cognome	Nome	Data di nascita
276545	Smith	Mary	25/11/1980
485745	Black	Anna	23/04/1981
200768	Verdi	Paolo	12/02/1981
587614	Smith	Lucy	10/10/1980
937653	Brown	Mavis	01/12/1980

Corsi			Esame		
Codice	Titolo	Tutor	Stud	Voto	Corso
01	Physics	Grant	276545	C	01
03	Chemistry	Beale	276545	B	04
04	Chemistry	Clark	937653	B	01

Notiamo che un esame è un oggetto che di per se non ha senso se non ci sono uno studente e un corso di cui ha sostenuto l'esame. La relazione, espressa dall'oggetto esame, tra lo studente e il corso è resa possibile attraverso i valori Stud e Corso.

Il vantaggio di utilizzare uno schema relazionale è che ci permette di avere sia l'indipendenza fisica che quella logica. Con indipendenza fisica si intende che il livello logico e quello esterno sono indipendenti da quello fisico. In questo modo una relazione è utilizzata nello stesso modo qualunque sia la sua realizzazione fisica e la realizzazione fisica può cambiare senza che debbano essere modificati i programmi. Con indipendenza logica si intende che il livello esterno è indipendente da quello logico, quindi aggiunte o modifiche alle viste non richiedono modifiche al livello logico e modifiche allo schema logico che lascino inalterato lo schema esterno sono trasparenti.

Definizione 2.0.1 – Dominio

Un *dominio* è un insieme possibilmente infinito di valori.

Siano $D_1, D_2, \dots, D_k$ domini, non necessariamente distinti, il prodotto cartesiano di tali domini, denotato da $D_1 \times D_2 \times \dots \times D_k$, è l'insieme:

$$\{(v_1, v_2, \dots, v_k) | v_1 \in D_1, v_2 \in D_2, \dots, v_k \in D_k\}$$

Una relazione matematica è un qualsiasi sottoinsieme del prodotto cartesiano di uno o più domini e una relazione che è sottoinsieme del prodotto cartesiano di k domini si dice di grado k . Gli elementi di una relazione sono detti tuple (oppure n -uple) e il numero di tuple di una relazione è la sua cardinalità. Le tuple di una relazione sono tutte distinte (almeno per un valore) ... dai vincoli sugli insiemi.

Esempio. Con $D_1 = \{a, b\}$ e $D_2 = \{x, y, z\}$ si ha:

$$D_1 \times D_2 = \{(a, x), (a, y), (a, z), (b, x), (b, y), (b, z), \}$$

Una relazione $r \subseteq D_1 \times D_2$ ad esempio $\{(a, x), (a, z), (b, y)\}$ è una relazione di grado 2 con cardinalità 3 e tuple $(a, x), (a, z), (b, y)$.

Notazione 2.0.1

r è una relazione di grado k , t è una tupla di r , i è un numero intero nell'insieme $\{1, \dots, k\}$ e $t[i]$ indica la i -esima componente di t .

Esempio.

0	a
0	c
1	b

Se $r = \{(0, a), (0, c), (1, b)\}$, $t = (0, a)$ è una tupla di r , $t[2] = a$, $t[1] = 0$, $t[1, 2] = (0, a)$.

Un attributo è definito da un nome A e dal dominio dell'attributo $dom(A)$. Sia R un insieme di attributi, un'ennupla su R è una funzione definita su R che associa ad ogni attributo A in R un elemento di $dom(A)$. Se t è un'ennupla su R ed A è un attributo in R , allora con $t(A)$ indicheremo il valore assunto dalla funzione t in corrispondenza dell'attributo A . L'insieme di attributi di una relazione è detto schema e uno schema di relazione $R(A_1, A_2, \dots, A_k)$ è invariante nel tempo e descrive la struttura dei nostri dati. L'istanza di una relazione con schema $R(X)$ contiene i valori attuali, che possono cambiare anche molto rapidamente nel tempo. Nel modello relazionale i riferimenti fra dati in relazioni diverse sono rappresentati per mezzo di valori dei domini che compaiono nelle ennuple.

I valori null rappresentano mancanza di informazione o il fatto che l'informazione non è applicabile, e sono valori polimorfi che non appartengono a nessun dominio ma possono sostituire i valori in qualsiasi dominio.

2.1 Vincoli di integrità

Un **vincolo di integrità** è una proprietà che deve essere soddisfatta da ogni istanza della base di dati, quindi è legata al suo schema. I vincoli descrivono proprietà specifiche del campo di applicazione, e quindi delle informazioni ad esso relative modellate attraverso la base di dati. Una istanza di base di dati è corretta se soddisfa tutti i vincoli di integrità associati al suo schema.

Esempio 2.1.1

Esami				Studenti		
Studente	Voto	Lode	Corso	Matricola	Cognome	Nome
276545	32		01	276545	Rossi	Mario
276545	30	si	02	787643	Neri	Piero
787643	27	si	03	787643	Bianchi	Luca
739430	24		04			

$(Voto \geq 18) \text{ AND } (Voto \leq 30)$ è un vincolo di dominio; $(Voto = 30) \text{ OR NOT } (Lode = "si")$ è un vincolo di tupla; Matricola unique è un vincolo tra tuple della stessa rela-

zione; `Studente references Studenti`. `Matricola` è un vincolo tra valori in tuple di relazioni diverse.

2.2 Chiavi

Le tuple di una relazione devono poter essere identificate univocamente.

Una **chiave** di una relazione è un attributo o insieme di attributi che identifica univocamente una tupla.

Quindi un insieme X di attributi di una relazione R è una chiave di R se soddisfa le seguenti condizioni:

1. Per ogni istanza di R , non esistono due tuple distinte t_1 e t_2 che hanno gli stessi valori per tutti gli attributi in X , cioè tali che $t_1[X] = t_2[X]$;
2. Nessun sottoinsieme proprio di X soddisfa la condizione 1.

Una relazione potrebbe avere anche più chiavi alternative e si sceglie quella più usata o quella composta da un numero minore di attributi come chiave primaria che non può essere nulla. Le chiavi consentono di mettere in relazione i dati in tabelle diverse.

Un vincolo di integrità referenziale (foreign key) esige che porzioni di informazione in relazioni diverse sono correlate attraverso valori di chiave. Ovvero, se una relazione fa riferimento al valore di un attributo o di un insieme di attributi che dovrebbe comparire in una seconda relazione, dobbiamo assicurarci che ciò avvenga realmente. Quindi un vincolo di integrità referenziale fra gli attributi X di una relazione R_1 e un'altra relazione R_2 impone ai valori su X in R_1 di comparire come valori della chiave primaria di R_2 .

2.3 Dipendenze funzionali

Una **dipendenza funzionale** stabilisce un particolare legame semantico tra due insiemi non vuoti di attributi X e Y appartenenti ad uno schema R . Tale vincolo si scrive $X \rightarrow Y$ e si legge X determina Y .

Esempio 2.3.1 – Dipendenze funzionali

Supponiamo di avere uno schema di relazione `Volì(CodiceVolo, Giorno, Pilota, Ora)` con i seguenti vincoli:

- Un volo con un certo codice parte sempre alla stessa ora;
- Esiste un solo volo con un dato pilota, in un dato giorno ad una data ora;
- C'è un solo pilota di un dato volo in un dato giorno.

Questi vincoli appena descritti corrispondono alle dipendenze funzionali:

- `CodiceVolo` $\rightarrow$ `Ora`
- `{Giorno, Pilota, Ora}` $\rightarrow$ `CodiceVolo`
- `{CodiceVolo, Giorno}` $\rightarrow$ `Pilota`

Diremo che una relazione r con schema R soddisfa la dipendenza funzionale $X \rightarrow Y$ se:

1. la dipendenza funzionale $X \rightarrow Y$ è applicabile ad R , nel senso che sia X sia il Y sono sottoinsiemi di R ;

2. le ennuple in r che concordano su X concordano anche su Y , cioè per ogni coppia di ennuple t_1 e t_2 in r :

$$t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$$

Osservazione. L'implicazione significa che se le tuple sono uguali su X , allora devono essere uguali anche su Y . Questo vuol dire che se non ci sono tuple uguali su X la condizione è comunque verificata.

Capitolo 3

Algebra relazionale

L'algebra relazionale è un linguaggio formale e procedurale. Formale perché ci permette di interrogare una base di dati relazionale con un insieme di operatori che possono essere applicati a una (operatori unari) o due (operatori binari) istanze di relazione e forniscono come risultato una nuova istanza. In particolare, individuiamo i seguenti quattro tipi di operatore:

1. Rimozione di specifiche parti di una relazione (Proiezione e Selezione);
2. Operazioni insiemistiche (Unione, Intersezione e Differenza);
3. Combinazione delle tuple di due relazioni (Prodotto cartesiano e Join);
4. Ridenominazione di attributi.

E' anche un linguaggio procedurale perché l'interrogazione consiste, in un'espressione in cui compaiono operatori dell'algebra e istanze di relazioni della base di dati, in una sequenza che prescrive in maniera puntuale l'ordine delle operazioni e i loro operandi.

3.1 Proiezione e Selezione

Una **proiezione** consente di effettuare un "taglio verticale" su una relazione, cioè di selezionare solo alcune colonne (attributi). Si denota con il simbolo π e $\pi_{A_1, A_2, \dots, A_k}(r)$ seleziona le colonne di r che corrispondono agli attributi $A_1, A_2, \dots, A_k$.

Esempio 3.1.1 – Proiezione

Supponiamo di voler ottenere una lista di tutti i clienti presenti in questa relazione:

Cliente		
Nome	C#	Città
Rossi	C1	Roma
Rossi	C2	Milano
Rossi	C3	Roma
Bianchi	C4	Roma
Verdi	C5	Roma

Effettuando la proiezione $\pi_{\text{Nome}}(\text{Cliente})$ otterremo come output:

$\pi_{\text{Nome}}(\text{Cliente})$
Rossi
Bianchi
Verdi

la proiezione elimina i duplicati e, in questo caso, risponde ad una richiesta diversa (la lista dei nomi diversi dei clienti). Per prevenire totalmente la perdita di informazione possiamo

sfruttare l'unicità della chiave C#, effettuando la proiezione $\pi_{\text{Nome}, \text{C\#}}(\text{Cliente})$, il cui output sarà:

$\pi_{\text{Nome}, \text{C\#}}(\text{Cliente})$	
Rossi	C1
Rossi	C2
Rossi	C3
Bianchi	C4
Verdi	C5

Una **selezione** consente di effettuare un “taglio orizzontale” su una relazione, cioè di selezionare solo le righe (tuple) che soddisfano una data condizione. Si denota con il simbolo σ e $\sigma_C(r)$ seleziona le tuple di r che soddisfano la condizione C .

La condizione di selezione è un'espressione booleana composta (tramite operatori $\wedge, \vee$ e $\neg$) in cui i termini semplici sono del tipo $A\theta B$ dove θ è un operatore di confronto ($<, =, >, \leq, \geq$) e A e B sono due attributi con lo stesso dominio ($\text{dom}(A) = \text{dom}(B)$).

Esempio 3.1.2 – Selezione

Vogliamo ottenere tutte le informazioni riguardo i clienti provenienti da Roma presenti in questa relazione:

Cliente		
Nome	C#	Città
Rossi	C1	Roma
Rossi	C2	Milano
Rossi	C3	Roma
Bianchi	C4	Roma
Verdi	C5	Roma

Possiamo effettuare una selezione $\sigma_{\text{Città}='Roma'}(\text{Cliente})$ per ottenere le tuple richieste:

$\sigma_{\text{Città}='Roma'}(\text{Cliente})$		
Rossi	C1	Roma
Rossi	C3	Roma
Bianchi	C4	Roma
Verdi	C5	Roma

Nel caso in cui vogliamo ottenere tutte le informazioni riguardo i clienti che si chiamano Rossi e provengono da Roma, possiamo effettuare una selezione $\sigma_{\text{Città}='Roma' \wedge \text{Nome}='Rossi'}(\text{Cliente})$ per ottenere le tuple richieste:

$\sigma_{\text{Città}='Roma' \wedge \text{Nome}='Rossi'}(\text{Cliente})$		
Rossi	C1	Roma
Rossi	C3	Roma

3.2 Unione, Differenza e Intersezione

L'**unione** consente di costruire una relazione contenente tutte le ennuple che appartengono ad almeno uno dei due operandi. Si denota con il simbolo $\cup$ e può essere applicata solo a operandi *union compatibili* cioè tali che:

- hanno lo stesso numero di attributi;
- gli attributi corrispondenti (nell'ordine) sono definiti sullo stesso dominio.

Da notare che non è necessario che gli attributi union compatibili abbiano lo stesso nome, ma ovviamente il risultato ha senso solo se hanno lo stesso significato.

Esempio 3.2.1 – Unione

Consideriamo le seguenti due relazioni, descriventi gli insegnanti e i responsabili di vari dipartimenti di una scuola ed effettuiamo la loro unione:

Docenti			Amministrativi		
Nome	CodDoc	Dipartimento	Nome	CodAmm	Dipartimento
Rossi	C1	Matematica	Esposito	C1	Lingue
Rossi	C2	Lettere	Riccio	C2	Matematica
Bianchi	C3	Matematica	Pierro	C3	Lettere
Verdi	C4	Lingue	Bianchi	C4	Lingue

Effettuando l'unione tra Docenti e Amministrativi, otteniamo una nuova relazione contenente tutti i membri dello staff:

Docenti $\cup$ Amministrativi		
Nome	Cod	Dipartimento
Rossi	C1	Matematica
Rossi	C2	Lettere
Bianchi	C3	Matematica
Verdi	C4	Lingue
Esposito	C1	Lingue
Riccio	C2	Matematica
Pierro	C3	Lettere
Bianchi	C4	Lingue

Esempio 3.2.2 – Unione

Consideriamo le seguenti due relazioni e applichiamo l'unione:

Docenti			Amministrativi			
Nome	CodDoc	Dipartimento	Nome	CodAmm	Dipartimento	Stip
Rossi	D1	Matematica	Esposito	A1	Lingue	1250
Rossi	D2	Lettere	Riccio	A2	Matematica	2000
Bianchi	D3	Matematica	Pierro	A3	Lettere	1000
Verdi	D4	Lingue				

In questo esempio non è possibile unire le due relazioni in quanto non sono union compatibili perché hanno un numero diverso di attributi. Per risolvere il problema, possiamo effettuare prima una proiezione su Amministrativi, per poi effettuare l'unione con Docenti:

Docenti $\cup \pi_{\text{Nome, CodAmm, Dipartimento}}(\text{Amministrativi})$		
Rossi	D1	Matematica
Rossi	D2	Lettere
Bianchi	D3	Matematica
Verdi	D4	Lingue
Esposito	A1	Lingue
Riccio	A2	Matematica
Pierro	A3	Lettere

La **differenza** consente di costruire una relazione contenente tutte le tuple che appartengono al primo operando e non appartengono al secondo. Si denota con il simbolo $-$ ed è applicabile solo a operandi union compatibili.

Esempio 3.2.3 – Differenza

Consideriamo le seguenti due relazioni, descriventi gli insegnanti e i responsabili di vari dipartimenti di una scuola ed applichiamo la differenza:

Docenti			Amministrativi		
Nome	CodFiscale	Dipartimento	Nome	CodFiscale	Dipartimento
Rossi	C1	Matematica	Esposito	C5	Lettere
Rossi	C2	Lettere	Riccio	C6	Matematica
Bianchi	C3	Matematica	Pierro	C7	Lingue
Verdi	C4	Lingue	Bianchi	C3	Matematica

Effettuando la differenza tra Docenti e Amministrativi, otteniamo una nuova relazione contenente tutti gli insegnanti che non sono responsabili. Analogamente, effettuando la differenza tra Amministrativi e Docenti, otteniamo una nuova relazione contenente tutti i responsabili che non sono insegnanti.

Docenti – Amministrativi			Amministrativi – Docenti		
Nome	CodFiscale	Dipartimento	Nome	CodFiscale	Dipartimento
Rossi	C1	Matematica	Esposito	C5	Lettere
Rossi	C2	Lettere	Riccio	C6	Matematica
Verdi	C4	Lingue	Pierro	C7	Lingue

L'**intersezione** consente di costruire una relazione contenente tutte le tuple che appartengono ad entrambi gli operandi. Si denota con il simbolo $\cap$ si applica solo a operandi union compatibili.

Esempio 3.2.4 – Intersezione

Consideriamo le seguenti due relazioni, descriventi gli insegnanti e i responsabili di vari dipartimenti di una scuola ed applichiamo l'intersezione:

Docenti			Amministrativi		
Nome	CodFiscale	Dipartimento	Nome	CodFiscale	Dipartimento
Rossi	C1	Matematica	Esposito	C5	Lettere
Rossi	C2	Lettere	Riccio	C6	Matematica
Bianchi	C3	Matematica	Pierro	C7	Lingue
Verdi	C4	Lingue	Bianchi	C3	Matematica

Effettuando l'intersezione tra Docenti e Amministrativi, otteniamo una nuova relazione contenente tutti gli insegnanti che sono anche responsabili:

Docenti $\cap$ Amministrativi		
Nome	CodFiscale	Dipartimento
Bianchi	C3	Matematica

3.3 Prodotto Cartesiano e Ridenominazione

Capita molto spesso che le informazioni per rispondere ad una interrogazione sono distribuite in più relazioni, in quanto coinvolgono più oggetti in qualche modo associati. Occorre, pertanto, individuare le relazioni in cui si trovano le informazioni che ci interessano, e combinare queste informazioni in maniera opportuna.

Il **prodotto cartesiano** consente di costruire una relazione contenente tutte le ennuple che si ottengono concatenando una ennupla del primo operando con una ennupla del secondo operando. Si denota con il simbolo $\times$ e si usa quando le informazioni per rispondere ad una query si trovano in relazioni diverse.

Per poter distinguere gli attributi con lo stesso nome nello schema risultante di un prodotto cartesiano possiamo usare l'operazione di **ridenominazione**, che si denota con il simbolo ρ .

Esempio 3.3.1 – Prodotto cartesiano e ridenominazione

Consideriamo le seguenti relazioni da cui vogliamo ottenere l'elenco di tutti i clienti e gli ordini da loro effettuati:

Cliente			Ordine			
Nome	C#	Città	O#	C#	A#	N-pezzi
Rossi	C1	Roma	O1	C1	A1	100
Rossi	C2	Milano	O2	C2	A2	200
Bianchi	C3	Roma	O3	C3	A2	150
Verdi	C4	Roma	O4	C4	A3	200
			O1	C1	A2	200
			O1	C1	A3	100

Prima di poter effettuare il prodotto cartesiano tra le due relazioni, è necessario ridenominare uno dei due attributi C# presenti in entrambe le relazioni:

$$\text{OrdineR} = \rho_{C\# \leftarrow CC\#}(\text{Ordini})$$

Successivamente, effettuando il prodotto cartesiano $\text{Cliente} \times \text{OrdineR}$, otteniamo:

Cliente $\times$ OrdineR						
Nome	C#	Città	O#	CC#	A#	N-pezzi
Rossi	C1	Roma	O1	C1	A1	100
Rossi	C1	Roma	O2	C2	A2	200
Rossi	C1	Roma	O3	C3	A2	150
Rossi	C1	Roma	O4	C4	A3	200
Rossi	C1	Roma	O1	C1	A2	200
Rossi	C2	Milano	O1	C1	A1	100
Rossi	C2	Milano	O2	C2	A2	200
...	...	...	...	...	...	...
...	...	...	...	...	...	...
Verdi	C4	Roma	O4	C4	A3	200
Verdi	C4	Roma	O1	C1	A2	200

A questo punto, però, notiamo la presenza di alcune incorrettezze. Ad esempio, il cliente "Rossi", avente codice C1, non ha mai effettuato l'ordine "(O2, C2, 200)", il quale invece è stato effettuato dal cliente avente codice C2. Possiamo risolvere tale problema effettuando una selezione dopo aver effettuato il prodotto cartesiano:

$\sigma_{C\#=CC\#}(\text{Cliente} \times \text{OrdineR})$						
Nome	C#	Città	O#	CC#	A#	N-pezzi
Rossi	C1	Roma	O1	C1	A1	100
Rossi	C1	Roma	O1	C1	A2	200
Rossi	C1	Roma	O1	C1	A3	100
Rossi	C2	Milano	O2	C2	A2	200
Bianchi	C3	Roma	O3	C3	A2	150
Verdi	C4	Roma	O4	C4	A3	200

Infine, per via del select svolto, le colonne C# e CC# risultano essere uguali, dunque potremmo rimuovere una delle due effettuando una proiezione. La query finale risulta essere:

$$\pi_{\text{Nome C# Città O# A# N-pezzi}}(\sigma_{\text{C\#}=\text{CC\#}}(\text{Cliente} \times \text{OrdineR}))$$

$\pi_{\text{Nome,C\#,Città,O\#,A\#,N-pezzi}}(\sigma_{\text{C\#}=\text{CC\#}}(\text{Cliente} \times \text{OrdineR}))$					
Nome	C#	Città	O#	A#	N-pezzi
Rossi	C1	Roma	O1	A1	100
Rossi	C1	Roma	O1	A2	200
Rossi	C1	Roma	O1	A3	100
Rossi	C2	Milano	O2	A2	200
Bianchi	C3	Roma	O3	A2	150
Verdi	C4	Roma	O4	A3	200

3.4 Join Naturale e Theta Join

Il **join naturale** consente di selezionare le tuple del prodotto cartesiano dei due operandi che soddisfano la condizione:

$$R_1A_1 = R_2A_1 \wedge R_1A_2 = R_2A_2 \wedge \dots \wedge R_1A_k = R_2A_k$$

dove R_1 ed R_2 sono i nomi degli schemi di relazioni e $A_1, A_2, \dots, A_k$ sono gli attributi comuni, cioè con lo stesso nome, delle relazioni. Il join naturale unisce le tuple delle istanze delle due relazioni che hanno gli stessi valori in corrispondenza degli attributi con la stessa etichetta. Si indica con il simbolo $\bowtie$:

$$r_1 \bowtie r_2 = \pi_{XY}(\sigma_C(r_1 \times r_2))$$

con $C = R_1A_1 = R_2A_1 \wedge \dots \wedge R_1A_k = R_2A_k$, X è l'insieme di attributi di r_1 e Y è l'insieme di attributi di r_2 che non sono attributi di r_1 .

Esempio 3.4.1 – Join naturale

Consideriamo le seguenti relazioni da cui vogliamo ottenere i nomi e le città dei clienti che hanno ordinato più di 100 pezzi per almeno un articolo con prezzo superiore a 2:

Cliente			Ordine				Articolo		
Nome	C#	Città	O#	C#	A#	N-pezzi	A#	Denom.	Prezzo
Rossi	C1	Roma	O1	C1	A1	100	A1	Piatto	3
Rossi	C2	Milano	O2	C2	A2	200	A2	Bicchiere	2
Bianchi	C3	Roma	O3	C3	A2	150	A3	Tazza	4
Verdi	C4	Roma	O4	C4	A3	200			
			O1	C1	A2	200			
			O1	C1	A3	100			

Prima di tutto, effettuiamo un join naturale tra le tre relazioni:

$$(\text{Cliente} \bowtie \text{Ordine}) \bowtie \text{Articolo}$$

$(\text{Cliente} \bowtie \text{Ordine}) \bowtie \text{Articolo}$							
Nome	C#	Città	O#	A#	N-pezzi	Denom.	Prezzo
Rossi	C1	Roma	O1	A1	100	Piatto	3
Rossi	C1	Roma	O1	A2	200	Bicchiere	2
Rossi	C1	Roma	O1	A3	100	Tazza	4
Rossi	C2	Milano	O2	A2	200	Bicchiere	2
Bianchi	C3	Roma	O3	A2	150	Bicchiere	2
Verdi	C4	Roma	O4	A3	200	Tazza	4

Successivamente, selezioniamo le tuple che rispettano le condizioni che ci interessano ed effettuiamo una proiezione sul nome e la città delle tuple ottenute:

$$\pi_{\text{Nome}, \text{Città}}(\sigma_{\text{N-pezzi} > 100 \wedge \text{Prezzo} > 2}((\text{Cliente} \bowtie \text{Ordine}) \bowtie \text{Articolo}))$$

$\pi_{\text{Nome}, \text{Città}}(\sigma_{\text{N-pezzi} > 100 \wedge \text{Prezzo} > 2}(\text{CliOrdArt}))$	
Nome	Città
Verdi	Roma

Proposizione 3.4.1 – Casi limite del join naturale

I casi limite del join naturale si verificano quando:

1. due relazioni hanno un insieme di attributi in comune ma nessun valore è uguale per tali attributi, e il risultato sarà un insieme vuoto;
2. due relazioni non hanno degli attributi in comune, e il join naturale degenera in un prodotto cartesiano.

Il **theta join** consente di selezionare le tuple del prodotto cartesiano dei due operandi che soddisfano una condizione del tipo $A \theta B$, dove θ è un operatore di confronto ($\theta \in \{<, =, >, \leq, \geq\}$), A è un attributo dello schema del primo operando, B è un attributo dello schema del secondo operando, con $\text{dom}(A) = \text{dom}(B)$.

Il theta join equivale a:

$$r_1 \bowtie_{A \theta B} r_2 = \sigma_{A \theta B}(r_1 \times r_2)$$

In alcuni casi può essere necessario effettuare il join tra una relazione e se stessa (self join), in modo da ottenere combinazioni di tuple della stessa relazione. Attenzione: utilizzare il join naturale al posto del theta join genererebbe una relazione identica a quella di partenza, poiché ogni tupla verrebbe joinata con se stessa scartando automaticamente gli attributi doppiati.

Esempio 3.4.2 – Theta join

Data la seguente relazione, vogliamo ottenere una lista dei codici e dei nomi dei dipendenti aventi un salario maggiore dei loro supervisori:

Impiegato				
Nome	C#	Sezione	Salario	Supervisore
Rossi	C1	B	100	C3
Pirlo	C2	A	200	C3
Bianchi	C3	A	500	NULL
Verdi	C4	B	200	C2
Neri	C5	B	150	C1
Tosi	C6	B	100	C1

In tal caso, la soluzione migliore risulta essere una selezione effettuata su self join di Impiegato, in modo da poter accoppiare ogni dipendente al suo supervisore. Quindi creiamo una copia della relazione per poi effettuare un theta join tra il codice dei dipendenti e il codice dei loro supervisori, specificando la relazione di appartenenza di ognuno dei due attributi confrontati:

$$\begin{aligned} \text{ImpiegatoC} &= \text{Impiegato} \\ \text{ImpSup} &= \text{ImpiegatoC} \bowtie_{\text{ImpiegatoC.Superiore} = \text{Impiegato.C\#Impiegato}} \end{aligned}$$

Successivamente, eseguiamo la selezione richiesta, mantenendo solo le tuple dove il salario del dipendente è maggiore del salario del suo supervisore:

$$\sigma_{\text{Impiegato.Salario} > \text{ImpiegatoC.Salario}}(\text{ImpSup})$$

$\sigma_{\text{Impiegato.Salario} > \text{ImpiegatoC.Salario}}(\text{ImpSup})$									
Nome	C#	Sezione	Salario	Supervisore	CNome	CC#	CSez	CSal	CSup
Verdi	C4	B	200	C2	Pirlo	C2	A	200	C3
Neri	C5	B	150	C1	Rossi	C1	B	100	C3
Tosi	C6	B	100	C1	Rossi	C1	B	100	C3

Infine, effettuiamo una proiezione sul nome e il codice del dipendente:

$\pi_{\text{Impiegato.Nome, Impiegato.C\#}}(\sigma_{\text{Impiegato.Salario} > \text{ImpiegatoC.Salario}}(\text{ImpSup}))$

ImpRes	
Nome	C#
Verdi	C4
Neri	C5
Tosi	C6

3.5 Quantificazione universale

Fino ad ora, abbiamo visto solo query inerenti la quantificazione esistenziale ($\exists$ - esiste almeno un), ossia la selezione di oggetti che soddisfino almeno una volta una determinata condizione.

Tuttavia, utilizzando solo gli operatori visti precedentemente, non abbiamo un modo per poter effettuare query inerenti alla quantificazione universale ($\forall$ - per ogni), ossia la selezione di oggetti che soddisfino sempre una determinata condizione.

Osservazione 3.5.1 – Quantificazione universale

Nella logica del primo ordine, la negazione di "Per ogni oggetto x la condizione è vera" corrisponde a "Esiste almeno un oggetto x per cui la condizione è falsa".

Per selezionare tutte le tuple di una relazione per cui una determinata condizione è sempre valida, quindi, ci basta scartare tutte le tuple per cui almeno una volta la condizione non è valida.

Esempio 3.5.1 – Quantificazione universale

Data la seguente relazione, vogliamo ottenere un elenco di tutti i nomi e la città di provenienza dei clienti che hanno sempre effettuato un ordine di più di 100 pezzi.

Cliente			Ordine		
Nome	C#	Città	C#	A#	N-pezzi
Rossi	C1	Roma	C1	A1	100
Rossi	C2	Milano	C2	A2	200
Bianchi	C3	Roma	C3	A2	150
Verdi	C4	Roma	C4	A3	200
			C1	A2	200
			C1	A3	100

Per ottenere l'elenco richiesto, ci basta scartare l'elenco dei nomi e delle città che almeno una volta non hanno acquistato più di 100 pezzi dall'elenco totale dei nomi e delle città:

$\text{ElencoNonValidi} = \sigma_{\neg(Qnty > 100)}(\text{Cliente} \bowtie \text{Ordine})$

ElencoNonValidi				
Nome	C#	Città	A#	N-pezzi
Rossi	C1	Roma	A1	100
Rossi	C1	Roma	A3	100

Ora possiamo effettuare la proiezione su nome e città eliminando quelli che non ci interessano:

$$R = \pi_{\text{Nome}, \text{Città}}(\text{Cliente} \bowtie \text{Ordine}) - \pi_{\text{Nome}, \text{Città}}(\text{ElencoNonValidi})$$

R	
Nome	Città
Bianchi	Roma
Verdi	Roma

Da notare che effettuiamo il join ($\pi_{\text{Nome}, \text{Città}}(\text{Cliente} \bowtie \text{Ordine})$) perché in Cliente potremmo avere clienti che non hanno mai effettuato ordini, che sarebbero preservati dalla sottrazione. Utilizzare il join assicura di effettuare la sottrazione a partire solo dai clienti che hanno effettuato almeno un ordine.

Esempio 3.5.2 – Quantificazione universale

Data la seguente relazione, vogliamo ottenere una lista dei codici e dei nomi dei capi aventi un salario maggiore di tutti i loro impiegati.

Impiegato				
Nome	C#	Dipart	Stip	Capo
Rossi	C1	B	100	C3
Pirlo	C2	A	200	C3
Bianchi	C3	A	500	NULL
Verdi	C4	B	200	C2
Neri	C5	B	150	C1
Tosi	C6	B	100	C1

Anche in questo caso, per ottenere l'elenco richiesto ci basta scartare i supervisor aventi il salario minore di almeno un dipendente:

$$\begin{aligned} \text{ImpiegatoC} &= \text{Impiegato} \\ \text{ImpSup} &= \text{ImpiegatoC} \bowtie \text{ImpiegatoC.Capo} = \text{Impiegato.C\#Impiegato} \\ \text{NonValidi} &= \pi_{\text{Nome}, \text{C\#}}(\sigma_{\neg(\text{Impiegato.Stip} < \text{ImpiegatoC.Stip})}(\text{ImpSup})) \\ R &= \pi_{\text{Nome}, \text{C\#}}(\text{ImpSup}) - \text{NonValidi} \end{aligned}$$

R	
Nome	C#
Bianchi	C3

Capitolo 4

Progettazione di una base di dati

4.1 Problemi e vincoli

Uno schema di dati ben progettato ha la premura di mantenere separati tra di loro concetti diversi. In caso contrario si possono presentare delle anomalie che rendono inconsistenti i nostri dati. Tra queste troviamo:

- *Ridondanza*, quando le informazioni si ripetono più volte nelle tabelle, con conseguente spreco di spazio e problemi di gestione dei dati;
- *Anomalie di aggiornamento*, quando un dato deve essere modificato più volte all'interno di una tabella per essere aggiornato;
- *Anomalie di inserimento*, quando alcuni campi non possono essere memorizzati;
- *Anomalie di cancellazione*, quando con l'eliminazione di alcuni dati si perdono delle informazioni importanti.

Quindi uno schema di basi di dati è ben progettato se non presenta ridondanze, anomalie di aggiornamento, inserimento e cancellazione.

Spesso la cattiva progettazione, cioè in particolare l'errore di rappresentare più concetti nella stessa relazione, deriva dalla necessità di eseguire frequentemente delle operazioni che fanno apparire più "comodo" memorizzare tutti i dati insieme. Abbiamo visto che le operazioni dell'algebra relazionale consentono, attraverso i riferimenti per valore tra gli oggetti e opportuni join tra relazioni, di ottenere le stesse informazioni senza incorrere nei problemi che abbiamo discusso.

Esempio 4.1.1 – Buona progettazione di una base di dati

Riprogettiamo la seguente relazione tenendo conto della separazione tra concetti diversi e delle anomalie che possono presentarsi.

Ordini						
Nome	C#	Città	A#	N-pezzi	Denom	Prezzo
Rossi	C1	Roma	A1	100	Piatto	3
Rossi	C2	Milano	A2	200	Bicchiere	2
Bianchi	C3	Roma	A2	150	Bicchiere	2
Verdi	C4	Roma	A3	200	Tazza	4
Rossi	C1	Roma	A2	200	Bicchiere	2
Rossi	C1	Roma	A3	100	Tazza	4

In questo schema abbiamo tutte le informazioni disponibili in un solo colpo, ma se bisogna aggiornare la città di un cliente, bisogna aggiornare tutte le tuple di quel cliente; se si cancella un articolo che va fuori produzione, si rischiano di perdere i dati di un cliente che l'ha ordinato; per inserire un articolo deve esserci per forza un cliente che lo ordina.

Un buon approccio è dividere i concetti di Cliente, Articolo e Ordine in tre relazioni separate.

Cliente			Articolo			Ordine		
Nome	C#	Città	A#	Denom	Prezzo	C#	A#	N-pezzi
Rossi	C1	Roma	A1	Piatto	3	C1	A1	100
Rossi	C2	Milano	A2	Bicchiere	2	C2	A2	200
Bianchi	C3	Roma	A3	Tazza	4	C3	A2	150
Verdi	C4	Roma				C4	A3	200
						C1	A2	200
						C1	A3	100

Per individuare i concetti rappresentati in una relazione si utilizza il concetto di chiave che determina una particolare dipendenza funzionale, che è un particolare tipo di vincolo.

4.2 Dipendenze funzionali

Un vincolo è la rappresentazione nello schema di una base di dati di una condizione valida nella realtà di interesse. Un'istanza della base di dati è legale se soddisfa tutti i vincoli (cioè se è una rappresentazione fedele della realtà). Un DBMS è dotato di procedure per la verifica dei vincoli (vincoli di dominio, chiavi, contenimento di domini) che ricorrono più frequentemente, per verificare che un'istanza della base di dati sia legale.

Le dipendenze funzionali definite su uno schema di relazione esprimono particolari vincoli di dipendenza tra sottoinsiemi di attributi dello schema stesso, che devono essere soddisfatti da ogni istanza dello schema.

Definizione 4.2.1 – Schema di relazione

Uno schema di relazione R è un insieme di attributi $\{A_1, A_2, \dots, A_n\}$

Definizione 4.2.2 – Tupla

Dato uno schema di relazione $R = A_1 A_2 \dots A_n$, una tupla t su R è una funzione che associa ad ogni attributo A_i in R un valore $t[A_i]$ nel corrispondente dominio $dom(A_i)$.

Se X è un sottoinsieme di R e t_1 e t_2 sono due tuple su R , t_1 e t_2 coincidono su X ($t_1[X] = t_2[X]$) se:

$$\forall A \in X (t_1[A] = t_2[A])$$

Definizione 4.2.3 – Istanza di relazione

Dato uno schema di relazione R , una istanza di R è un insieme di tuple su R .

Tutte le "tabelle" che abbiamo visto finora negli esempi sono istanze di qualche schema di relazione.

Dato uno schema di relazione R , una **dipendenza funzionale** su R è una coppia ordinata di sottoinsiemi non vuoti X ed Y di R . Si indica con $X \rightarrow Y$ e vuol dire che X determina funzionalmente Y , oppure, Y dipende funzionalmente da X .

Dati uno schema R e una dipendenza funzionale $X \rightarrow Y$ su R , un'istanza r di R soddisfa la dipendenza funzionale $X \rightarrow Y$ se:

$$\forall t_1, t_2 \in r (t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y])$$

Osservazione. Ovviamente se $t_1[X] \neq t_2[X]$ la dipendenza è soddisfatta qualunque siano i valori di $t_1[Y]$ e $t_2[Y]$.

Da notare che le dipendenze funzionali non fanno altro che esprimere dei vincoli di sui dati. Ad esempio se due tuple hanno uguale codice fiscale (si riferiscono alla stessa persona) devono avere uguale anche la data di nascita.

Esempio 4.2.1 – Dipendenze funzionali

Consideriamo le seguenti relazioni:

R_1				R_2			
A	B	C	D	A	B	C	D
a1	b1	c1	d1	a1	b1	c1	d1
a1	b2	c1	d2	a1	b1	c2	d2
a1	b1	c1	d3	a1	b2	c1	d3

La relazione R_1 soddisfa la dipendenza funzionale $AB \rightarrow C$ per la prima e la terza tupla. La relazione R_2 , invece, non soddisfa la dipendenza funzionale $AB \rightarrow C$ per la prima e seconda tupla.

Dati uno schema di relazione R e un insieme F di dipendenze funzionali, un'istanza di R è **legale** se soddisfa tutte le dipendenze in F .

Esempio 4.2.2 – Istanza legale

Consideriamo la seguente relazione su cui sono definite le seguenti dipendenze funzionali:

$$F\{A \rightarrow B, B \rightarrow C\}$$

A	B	C	D
a1	b1	c1	d1
a1	b1	c1	d2
a2	b2	c1	d3

Tale istanza è legale su F , poiché soddisfa le dipendenze funzionali in F : tutte le tuple aventi $t[A] = a1$ hanno anche $t[B] = b1$ e tutte le tuple aventi $t[B] = b1$ hanno anche $t[C] = c1$. In realtà, indirettamente, soddisfa anche la dipendenza $A \rightarrow C$ che può essere considerata come se fosse in F .

4.2.1 Chiusura di F

Dato uno schema di relazione R e un insieme F di dipendenze funzionali su R la **chiusura di F** è l'insieme delle dipendenze funzionali che sono soddisfatte da ogni istanza legale di R . Si indica con F^+ e banalmente si ha che $F \subseteq F^+$.

Definizione 4.2.4 – Chiave

Dati uno schema di relazione R e un insieme F di dipendenze funzionali un sottoinsieme K di uno schema di relazione R è una **chiave** di R se:

1. $K \rightarrow R \in F^+$
2. non esiste un sottoinsieme proprio K' di K tale che $K' \rightarrow R \in F$.

Osservazione 4.2.1 – Dipendenze funzionali banali

Dati uno schema di relazione R e due sottoinsiemi non vuoti X, Y di R tali che $Y \subseteq X$ si ha che ogni istanza r di R soddisfa la dipendenza funzionale $X \rightarrow Y$. Pertanto, la dipendenza funzionale $Y \subseteq X \quad X \rightarrow Y \in F^+$ è detta banale.

Proposizione 4.2.1 – Proprietà delle dipendenze funzionali

Dati uno schema di relazione R e un insieme di dipendenze funzionali F , si ha:

$$X \rightarrow Y \in F^+ \iff \forall A \in Y (X \rightarrow A \in F^+)$$

$X \rightarrow Y$ deve essere soddisfatta da ogni istanza legale di R e:

- Se $t_1[X] = t_2[X]$ allora deve essere $t_1[Y] = t_2[Y]$;
- Se $A \in Y$ e $t_1[A] \neq t_2[A]$, non può essere $t_1[Y] = t_2[Y]$;
- Se $\forall A \in Y t_1[A] = t_2[A]$, avremo $t_1[Y] = t_2[Y]$.

4.3 Assiomi di Armstrong

Il nostro problema è calcolare l'insieme di dipendenze F^+ che vengono soddisfatte da ogni istanza legale di uno schema R su cui è definito un insieme di dipendenze funzionali F . Abbiamo concluso che banalmente $F \subseteq F^+$ e possiamo introdurre un nuovo insieme che chiameremo F^A .

Definiamo come F^A l'insieme di tutte le dipendenze funzionali ottenibili partendo da F e applicando i seguenti **assiomi di Armstrong**:

- *Inclusione iniziale*:

$$f \in F \implies f \in F^A$$

- *Assioma della riflessività*:

$$Y \subseteq X \subseteq R \implies X \rightarrow Y \in F^A$$

- *Assioma dell'aumento*:

$$X \rightarrow Y \in F^A \implies XZ \rightarrow YZ \in F^A, \forall Z \subseteq R$$

- *Assioma della transitività*:

$$X \rightarrow Y \in F^A, Y \rightarrow Z \in F^A \implies X \rightarrow Z \in F^A$$

Proposizione 4.3.1 – Regole secondarie di Armstrong

Tramite gli assiomi di Armstrong è possibile ricavare le seguenti regole aggiuntive:

- *Regola dell'unione*:

$$X \rightarrow Y \in F^A, X \rightarrow Z \in F^A \implies X \rightarrow YZ \in F^A$$

- *Regola della decomposizione*:

$$X \rightarrow Y \in F^A, Z \subseteq Y \implies X \rightarrow Z \in F^A$$

- *Regola della pseudo-transitività*:

$$X \rightarrow Y \in F^A, WY \rightarrow Z \in F^A \implies WX \rightarrow Z \in F^A$$

Dimostrazione.

- Se $X \rightarrow Y \in F^A$, per l'assioma dell'aumento si ha $X \rightarrow XY \in F^A$. Analogamente, se $X \rightarrow Z \in F^A$, per l'assioma dell'aumento si ha $XY \rightarrow YZ \in F^A$. Quindi, poiché $X \rightarrow XY \in F^A$ e $XY \rightarrow YZ \in F^A$, per l'assioma della transitività si ha $X \rightarrow YZ \in F^A$.

- Se $Z \subseteq Y$ allora, per l'assioma della riflessività, si ha $Y \rightarrow Z \in F^A$. Quindi, poiché $X \rightarrow Y \in F^A$ e $Y \rightarrow Z \in F^A$, per l'assioma della transitività si ha $X \rightarrow Z \in F^A$.
- Se $X \rightarrow Y \in F^A$, per l'assioma dell'aumento si ha $WX \rightarrow WY \in F^A$. Quindi, poiché $WX \rightarrow WY \in F^A$ e $WY \rightarrow Z \in F^A$, per l'assioma della transitività si ha $WX \rightarrow Z \in F^A$.

Proposizione 4.3.2

Dato uno schema R e un insieme F di dipendenze funzionali definite su R , si ha che:

$$X \rightarrow Y \in F^A \iff \forall A \in Y, X \rightarrow A \in F^A$$

Dimostrazione. Siano $X, Y \subseteq R$, dove $Y = \{A_1, \dots, A_k\}$. Per la regola dell'unione, si ha che:

$$\forall A_i \in Y, X \rightarrow A_i \in F^A \implies X \rightarrow \{A_1, \dots, A_k\} = Y \in F^A$$

Per la regola della decomposizione, invece si ha che:

$$X \rightarrow Y \in F^A \implies \forall A \in Y, X \rightarrow A \in F^A$$

4.3.1 Chiusura di un insieme di attributi

Siano R uno schema di relazione, F un insieme di dipendenze funzionali su R e X un sottoinsieme di R . La **chiusura di X rispetto ad F** , denotata con X_F^+ (o semplicemente X^+ , se F è l'unico insieme di dipendenze su R) è definito da:

$$X_F^+ = \{A \in R \mid X \rightarrow A \in F^A\}$$

In pratica fanno parte della chiusura di un insieme di attributi X tutti quelli che sono determinati funzionalmente da X eventualmente applicando gli assiomi di Armstrong.

Lemma 4.3.1 – Relazione tra F^+ e X^+

Siano R uno schema di relazione ed F un insieme di dipendenze funzionali su R . Si ha che:

$$X \rightarrow Y \in F^A \iff Y \subseteq X^+$$

Dimostrazione. Sia $Y = A_1, A_2, \dots, A_n$, poiché $Y \subseteq X^+$, $\forall i = 1, \dots, n$, si ha che $X \rightarrow A_i \in F^A$. Pertanto, per la regola dell'unione, $X \rightarrow Y \in F^A$. Poiché $X \rightarrow Y \in F^A$, per la regola della decomposizione si ha che, $\forall i = 1, \dots, n$, $X \rightarrow A_i \in F^A$, cioè $A_i \in X^+$, $\forall i = 1, \dots, n$, e, quindi, $Y \subseteq X^+$.

4.3.2 $F^+ = F^A$ **Teorema 4.3.1 – $F^+ = F^A$**

Siano R uno schema di relazione ed F un insieme di dipendenze funzionali su R . Si ha:

$$F^+ = F^A$$

Dimostrazione. $F^+ = F^A$ vuol dire che:

$$\begin{aligned} \textcircled{1} \quad F^A &\subseteq F^+ & X \rightarrow Y \in F^A &\implies X \rightarrow Y \in F^+ \\ \textcircled{2} \quad F^+ &\subseteq F^A & X \rightarrow Y \in F^+ &\implies X \rightarrow Y \in F^A \end{aligned}$$

1. Per provare che $F^A \subseteq F^+$ dimostriamo che $X \rightarrow Y \in F^A$ per induzione sul numero i di applicazioni di uno degli assiomi di Armstrong.

Per il *caso base* ($i = 0$) non abbiamo applicato alcun assioma di Armstrong, allora l'unica possibilità è che:

$$X \rightarrow Y \in F \implies X \rightarrow Y \in F^+$$

Per *ipotesi induttiva* abbiamo che ogni dipendenza funzionale ottenuta a partire da F applicando gli assiomi di Armstrong un numero di volte minore o uguale a $i - 1$ è in F^+ :

$$X \rightarrow Y \in F^A \text{ tramite } i - 1 \text{ assiomi} \implies X \rightarrow Y \in F^+$$

Per il *passo induttivo* è necessario dimostrare che se $X \rightarrow Y \in F^A$ dopo aver applicato $i + 1$ assiomi di Armstrong, allora $X \rightarrow Y \in F^+$. E' possibile ritrovarsi in uno dei seguenti tre casi:

- (a) se $X \rightarrow Y$ è stata ottenuta mediante l'*assioma della riflessività*, in tal caso $Y \subseteq X$. Sia r un'istanza di R e siano t_1 e t_2 due tuple di r tali che $t_1[X] = t_2[X]$; banalmente si ha $t_1[Y] = t_2[Y]$.
 - (b) se $X \rightarrow Y$ è stata ottenuta applicando l'*assioma dell'aumento* ad una dipendenza funzionale $V \rightarrow W \in F^A$, con $X = VZ$ e $Y = WZ$, ottenuta a sua volta applicando ricorsivamente gli assiomi di Armstrong un numero di volte minore o uguale a $i - 1$ (quindi per l'ipotesi induttiva $V \rightarrow W \in F^+$); sarà quindi $X \rightarrow VZ$ e $Y \rightarrow WZ$, per qualche $Z \subseteq R$. Sia r un'istanza legale di R e siano t_1 e t_2 due tuple di r tali che $t_1[X] = t_2[X]$; banalmente si ha che $t_1[VZ] = t_2[VZ] \implies t_1[V] = t_2[V] \wedge t_1[Z] = t_2[Z]$. Per l'ipotesi induttiva da $t_1[V] = t_2[V]$ segue $t_1[W] = t_2[W]$; da $t_1[W] = t_2[W]$ e $t_1[Z] = t_2[Z]$ segue $t_1[Y] = t_2[Y]$.
 - (c) se $X \rightarrow Y$ è stata ottenuta applicando l'*assioma della transitività* a due dipendenze funzionali $X \rightarrow Z$ e $Z \rightarrow Y \in F^A$, ottenute a loro volta applicando ricorsivamente gli assiomi di Armstrong un numero di volte minore o uguale a $i - 1$ (quindi per l'ipotesi induttiva $X \rightarrow Z$ e $Z \rightarrow Y \in F^+$). Sia r un'istanza legale di R e siano t_1 e t_2 due tuple di r tali che $t_1[X] = t_2[X]$. Per l'ipotesi induttiva da $t_1[X] = t_2[X]$ segue $t_1[Z] = t_2[Z]$; da $t_1[Z] = t_2[Z]$, ancora per l'ipotesi induttiva segue $t_1[Y] = t_2[Y]$.
2. Per provare che $F^+ \subseteq F^A$ supponiamo per assurdo che esista una dipendenza funzionale $X \rightarrow Y \in F^+$ tale che $X \rightarrow Y \notin F^A$. Useremo una particolare istanza legale di R per dimostrare che questa supposizione porta ad una contraddizione.

Consideriamo la seguente istanza r di R :

r							
X^+				$R - X^+$			
1	1	...	1	1	1	...	1
1	1	...	1	0	0	...	0

L'istanza da solo due tuple, uguali sugli attributi in X^+ e diverse in tutti gli altri $R - X^+$. Dimosteremo che (a) l'istanza è legale e che (b) se $X \rightarrow Y \in F^+$ avremo necessariamente che $X \rightarrow Y \in F^A$.

- (a) Sia $V \rightarrow W$ una dipendenza funzionale in F e se le due tuple sono diverse su V , allora la dipendenza è soddisfatta, se hanno gli stessi valori per V allora $V \subseteq X^+$, perché le tuple sono uguali solo per quell'insieme di attributi; per il Lemma 4.3.1, si ha che $X \rightarrow V \in F^A$; pertanto, per l'assioma della transitività ($X \rightarrow V$ e $V \rightarrow W$), $X \rightarrow W \in F^A$ e, quindi, di nuovo per il Lemma 4.3.1, $W \subseteq X^+$ quindi le due tuple sono uguali anche sui valori di W . Quindi $V \rightarrow W$ è soddisfatta e poiché abbiamo considerato una qualunque dipendenza in F , r le soddisfa tutte, quindi è legale.
- (b) Supponiamo per assurdo che $X \rightarrow Y \in F^+$ e $X \rightarrow Y \notin F^A$. Abbiamo mostrato che la nostra r è un'istanza legale. Poiché le dipendenze in F^+ sono soddisfatte da ogni istanza legale, allora r soddisfa $X \rightarrow Y \in F^+$. Abbiamo due tuple uguali su X poiché $X \subseteq X^+$ (per l'assioma della riflessività e il Lemma 4.3.1) le due tuple di r coincidono sugli attributi X e quindi, poiché r soddisfa $X \rightarrow Y$, devono coincidere anche sugli attributi di Y . Questo implica che $Y \subseteq X^+$ e, per il Lemma 4.3.1, che $X \rightarrow Y \in F^A$.

E' utile notare che la dimostrazione di questo teorema si basa su due collegamenti molto importanti:

- Il collegamento che esiste tra l'insieme di dipendenze F^+ e le istanze legali: se una istanza è legale allora soddisfa anche tutte le dipendenze in F^+ e d'altra parte F^+ è l'insieme di dipendenze soddisfatte da ogni istanza legale.
- Il collegamento che esiste tra la chiusura X^+ di un insieme di attributi X e il sottoinsieme di dipendenze in F^A (che abbiamo appena visto essere uguale a F^+) che hanno X come determinante, cioè $Y \subseteq X^+ \iff X \rightarrow Y \in F^A$ che equivale a dire che $Y \subseteq X^+ \iff X \rightarrow Y \in F^+$ e in particolare che $X \rightarrow Y$ deve essere soddisfatta da ogni istanza legale.

Quindi ora abbiamo un modo per identificare tutte le dipendenze in F^+ partendo da F e applicando gli assiomi di Armstrong e le regole derivate. Tuttavia, calcolare $F^+ = F^A$ richiede tempo esponenziale dato che ogni possibile sottoinsieme di R genera una dipendenza.

Quindi parleremo di terza forma normale (3NF), delle sue proprietà e di come ottenerla dallo schema di relazione che abbiamo progettato.

Capitolo 5

Terza forma normale (3NF)

Dati uno schema di relazione R e un insieme di dipendenze funzionali F su R , R è in **terza forma normale** se:

$$\forall X \rightarrow A \in F^+, A \neq X$$

allora, o A appartiene ad una chiave (è primo), oppure X contiene una chiave (è una superchiave).

Abbiamo già notato alcune proprietà delle dipendenze funzionali anche prima di formalizzarle con gli assiomi di Armstrong (vedi Osservazione 4.2.1 e Proposizione 4.2.1).

Esempio 5.0.1 – Terza forma normale

Sia $R = ABCD$ uno schema e sia $F = \{A \rightarrow B, B \rightarrow CD\}$ un insieme di dipendenze funzionali su R .

Applicando gli assiomi di Armstrong, si ha che:

Per riflessività:

$$A \subseteq A \implies A \rightarrow A \in F^A$$

Per transitività:

$$A \rightarrow B, B \rightarrow CD \in F^A \implies A \rightarrow CD \in F^A$$

Per unione:

$$A \rightarrow A, A \rightarrow B, A \rightarrow CD \in F^A \implies A \rightarrow ABCD = R \in F^A$$

Dunque, siccome $A \rightarrow R \in F^A = F^+$ e siccome A non ha sottoinsiemi, allora A è chiave di R (in particolare, A è l'unica chiave di R). Verifichiamo quindi se R sia in 3NF:

$A \rightarrow B \in F^+$ rispetta la definizione di 3NF, poiché il determinante A è chiave (e quindi anche una superchiave di se stessa). $B \rightarrow CD \in F^+$ non è una dipendenza da controllare, poiché CD sono un sottoinsieme di attributi e non un singolo attributo. Tuttavia, per decomposizione abbiamo che $B \rightarrow CD \in F^A = F^+ \implies B \rightarrow C, B \rightarrow D \in F^A = F^+$. Per entrambe si ha che B non è superchiave, poiché $A \not\subseteq B$, mentre C e D non sono primi, poiché $C, D \notin A$, dunque concludiamo che R non sia in 3NF.

Esempio 5.0.2 – Terza forma normale

Sia $R = ABCD$ e sia $F = \{AC \rightarrow B, B \rightarrow AD\}$ un insieme di dipendenze funzionali su R .

Applicando gli assiomi di Armstrong, si ha che:

Per riflessività:

$$A, C, AC \subseteq AC \implies AC \rightarrow A, AC \rightarrow C, AC \rightarrow AC \in F^A$$

$$B, C, BC \subseteq BC \implies BC \rightarrow B, BC \rightarrow C, BC \rightarrow BC \in F^A$$

Per transitività:

$$\begin{aligned} AC \rightarrow B, B \rightarrow AD \in F^A &\implies AC \rightarrow AD \in F^A \\ BC \rightarrow B, B \rightarrow AD \in F^A &\implies BC \rightarrow AD \in F^A \end{aligned}$$

Per unione:

$$\begin{aligned} AC \rightarrow C, AC \rightarrow B, AC \rightarrow AD \in F^A &\implies AC \rightarrow ABCD = R \in F^A \\ BC \rightarrow B, BC \rightarrow C, BC \rightarrow AD \in F^A &\implies BC \rightarrow ABCD \in F^A \end{aligned}$$

Per aumento:

$$AC \rightarrow ABCD = R \in F^A \implies ABC \rightarrow ABBCD = ABCD = R \in F^A$$

Deduciamo quindi che AC e BC siano chiave di R , mentre ABC è una superchiave di R . Verifichiamo quindi se R sia in 3NF:

$AC \rightarrow B \in F^A = F^+$ rispetta la definizione di 3NF, poiché AC è chiave. $B \rightarrow AD \in F^A = F^+$ non va controllato, ma per decomposizione si ha che $B \rightarrow A, B \rightarrow D \in F^A = F^+$. $B \rightarrow A \in F^+$ rispetta la definizione di 3NF, poiché $A \in AC$ e dunque primo, mentre $B \rightarrow D \in F^+$ non rispetta la definizione di 3NF, poiché né B è superchiave né D è primo, dunque concludiamo che R non sia in 3NF.

Esempio 5.0.3 – Terza forma normale

Sia $R = ABCD$ uno schema e sia $F = \{AB \rightarrow CD, BC \rightarrow A, D \rightarrow AC\}$ un insieme di dipendenze funzionali su R .

In tal caso si ha che AB , BC e BD sono chiavi di R . Verifichiamo quindi se R sia in 3NF:

$AB \rightarrow CD \in F^+$ rispetta la definizione di 3NF, poiché AB è chiave. $BC \rightarrow A \in F^+$ rispetta la definizione di 3NF, poiché BC è chiave e A è primo. $D \rightarrow AC \in F^+ \implies D \rightarrow A, D \rightarrow C \in F^+$, i quali rispettano entrambi la definizione di 3NF, poiché A e C sono entrambi primi. Dunque, concludiamo che R sia in 3NF.

5.1 Dipendenze parziali e transitive

Definizione 5.1.1 – Dipendenza parziale

Sia R uno schema e sia F un insieme di dipendenze funzionali definite su R . Definiamo $X \rightarrow A \in F^+$, dove $A \notin X$, come dipendenza parziale su R se:

- A non è primo
- $\exists K \subseteq R$ chiave di R tale che $X \subset K$ (X è contenuto in una chiave di R)

Definizione 5.1.2 – Dipendenza transitiva

Sia R uno schema e sia F un insieme di dipendenze funzionali definite su R . Definiamo $X \rightarrow A \in F^+$, dove $A \notin X$, come dipendenza transitiva su R se:

- A non è primo
- $\forall K \subseteq R$ chiave di R si ha che $X \not\subseteq K$ e $K - X \neq \emptyset$

Esempio 5.1.1 – Dipendenze parziali e transitive

Consideriamo il seguente schema:

Studente(Mat_r, CF, Cogn, Nome, DataN, Com, Prov, C#, Tit, Doc, DataE, Voto)

La dipendenza funzionale $\text{Matr} \rightarrow \text{Cogn}$ è una conseguenza della dipendenza funzionale $\text{Matr} \rightarrow \text{Cogn}$ che viene detta dipendenza parziale e si dice che l'attributo Cogn dipende parzialmente dalla chiave Matr .

La dipendenza funzionale $\text{Matr} \rightarrow \text{Prov}$ è una conseguenza delle due dipendenze funzionali $\text{Matr} \rightarrow \text{Com}$ e $\text{Com} \rightarrow \text{Prov}$. $\text{Com} \rightarrow \text{Prov}$ viene detta dipendenza transitiva e si dice che l'attributo Prov dipende transitivamente dalla chiave Matr .

Teorema 5.1.1 – Definizione alternativa di 3NF

Siano R uno schema di relazione e F un insieme di dipendenze funzionali su R . Uno schema R è in 3NF se e solo se non esistono né dipendenze parziali né dipendenze transitive in R .

Dimostrazione. Se non esistono né dipendenze parziali né dipendenze transitive in R supponiamo per assurdo che R non sia in 3NF; in tal caso esiste una dipendenza funzionale $X \rightarrow A \in F^+$ t.c. A non è primo e X non è una superchiave. Poiché X non è una superchiave sono possibili due casi:

- per ogni chiave K di R X non è contenuto propriamente in K e $K - X \neq \emptyset$; in tal caso $X \rightarrow A$ è una dipendenza transitiva su R (contraddizione);
- esiste una chiave K di R tale che $X \subset K$; in tal caso $X \rightarrow A$ è una dipendenza parziale su R (contraddizione).

5.2 Forma normale di Boyce-Codd

Definizione 5.2.1 – Forma normale di Boyce-Codd

Una relazione è in forma normale di Boyce-Codd (BCNF, Boyce-Codd Normal Form) quando in essa ogni determinante è una superchiave (ricordiamo che una chiave è anche superchiave). Una relazione che rispetta la forma normale di Boyce-Codd è anche in terza forma normale, ma non è vero l'opposto.

Può non essere possibile decomporre uno schema non BCNF ottenendo sottoschemi BCNF e preservando allo stesso tempo tutte le dipendenze. Ma questo è sempre possibile per la 3NF che è comunque soddisfacente, quindi nel seguito continueremo a prendere in considerazione solo la 3NF.

5.3 Chiusura di un insieme di attributi

Per il calcolo della chiusura dell'insieme di attributi X , denotata con X^+ , possiamo usare il seguente algoritmo.

Algoritmo 5.3.1 – Algoritmo per la chiusura di un insieme di attributi

Dato un qualsiasi insieme di attributi X , sottoinsieme di R , è possibile calcolare tutti gli elementi appartenenti a X^+ :

```
begin
  Z := X
  S := {A | Y → V ∈ F ∧ A ∈ V ∧ Y ⊆ Z}
  while S ⊄ Z do:
    Z := Z ∪ S
    S := {A | Y → V ∈ F ∧ A ∈ V ∧ Y ⊆ Z}
  end
  X := Z
end
```

Ovvero, si inseriscono in S i singoli attributi che compongono le parti destre di dipendenze in F la cui parte sinistra è contenuta in Z . All'inizio Z è proprio X , quindi inseriamo gli attributi che sono determinati funzionalmente da X ; una volta che questi sono entrati in Z , da questi ne aggiungiamo altri per transitività. Notiamo come alla fine di ogni iterazione aggiungiamo nuovi attributi a Z ; l'algoritmo si ferma quando il nuovo insieme S che otteniamo è già contenuto nell'insieme Z , cioè quando non possiamo aggiungere nuovi attributi alla chiusura transitiva di X .

Teorema 5.3.1 – Algoritmo per la chiusura di un insieme di attributi

Sia R uno schema e sia F un insieme di dipendenze funzionali definite su R . Dato un qualsiasi insieme di attributi $X \subseteq R$, l'algoritmo per il calcolo della chiusura di un insieme di attributi restituisce X^+ .

Dimostrazione. Indichiamo con Z_0 il valore iniziale di Z ($Z_0 = X$) e con Z_i ed S_i , $i \geq 1$, i valori di Z ed S dopo l' i -esima esecuzione del corpo del ciclo; è facile vedere che $Z_i \subseteq Z_{i+1}$, per ogni i .

Sia j tale che $S_j \subseteq Z_j$ (cioè, Z_j è il valore di Z quando l'algoritmo termina); proveremo che:

$$A \in Z_j \iff A \in X^+$$

1. Dimostriamo quindi per induzione che $Z_j \subseteq X^+$:

Per il *caso base* ($i = 0$) si ha alla 0-esima iterazione del while (ossia prima di esso) si ha $Z_0 = X \subseteq X^+$;

L'*ipotesi induttiva* si ha che, per ogni i , $Z_i \subseteq X^+$

Per il *passo induttivo*, sia A un attributo in $Z_i - Z_{i-1}$; deve esistere una dipendenza $Y \rightarrow V \in F$ tale che $Y \subseteq Z_{i-1}$ e $A \in V$. Poiché $Y \subseteq Z_{i-1}$, per l'ipotesi induttiva si ha che $Y \subseteq X^+$; pertanto, per il Lemma 4.3.1, $X \rightarrow Y \in F^A$. Poiché $X \rightarrow Y \in F^A$ e $Y \rightarrow V \in F$, per l'assioma della transitività si ha $X \rightarrow V \in F^A$ e quindi, per il Lemma 4.3.1, $V \subseteq X^+$. Pertanto, per ogni $A \in Z_i - Z_{i-1}$ si ha $A \in X^+$. Da ciò segue, per l'ipotesi induttiva, che $Z_i \subseteq X^+$.

2. Dimostriamo ora che $X^+ \subseteq Z_j$:

Sia $X \subseteq R$ e sia r istanza di $R(Z_j, R - Z_j)$ tale che:

r							
Z_j				$R - Z_j$			
1	1	...	1	1	1	...	1
1	1	...	1	0	0	...	0

vogliamo dimostrare che: (a) r è un'istanza legale e (b) $A \in Z_j$.

- (a) Data una qualunque dipendenza funzionale $V \rightarrow W$ in F : se $V \not\subseteq Z_j$ allora le due tuple sono diverse su V , quindi la dipendenza è soddisfatta; se $V \subseteq Z_j$ i valori delle due tuple sono uguali e se le due tuple avessero valori diversi su W , cioè se $W \cap (R - Z_j) \neq \emptyset$ si avrebbe $S_j \not\subseteq Z_j$ e quindi Z_j non sarebbe il valore finale di Z (contraddizione).
- (b) Poiché r è un'istanza legale di R deve soddisfare $X \rightarrow A$. Poiché $X = Z_0 \subseteq Z_j$, ne segue che la dipendenza non può essere soddisfatta a vuoto, dunque, l'unica possibilità affinché $X \rightarrow A \in F^+$ sia soddisfatta da r è $A \in Z_j$.

Dunque, siccome $A \in X^+ \implies A \in Z_j$, concludiamo che $X^+ \subseteq Z_j$.

Esempio 5.3.1 – Algoritmo per la chiusura di un insieme di attributi

Dato lo schema di relazione $R = (A, B, C, D, E, H)$ e il seguente insieme di dipendenze funzionali su $F = \{AB \rightarrow CD, EH \rightarrow D, D \rightarrow H\}$ su R , calcolare le chiusure di A , D e AB .

1. Applicando l'algoritmo otteniamo $Z := A$ e $S = \emptyset$ perché A da solo non determina alcun altro attributo. Dal controllo del while risulta che $\emptyset \subset A$ quindi non entriamo nel ciclo e:

$$A^+ = A$$

2. Applicando l'algoritmo otteniamo $Z := D$ e $S := H$ (per la dipendenza $D \rightarrow H$). $H \not\subset D$, quindi viene eseguito il while e Z diventa $Z \cup S = D \cup H = DH$, mentre S rimane H . Ora dato che $H \subset DH$ si esce dal ciclo con risultato:

$$D^+ = DH$$

3. Applicando l'algoritmo otteniamo $Z := AB$ e $S := CD$ (per la dipendenza $AB \rightarrow CD$). $CD \not\subset AB$, quindi viene eseguito il while e Z diventa $Z \cup S = AB \cup CD = ABCD$, mentre S diventa CDH (per la dipendenza $D \rightarrow H$). $CDH \not\subset ABCD$, quindi viene eseguito un altro ciclo con Z che diventa $Z \cup S = ABCD \cup CDH = ABCDH$, mentre S rimane CDH ($EH \rightarrow D$ non si può aggiungere a S perché $E \notin Z$). Ora abbiamo che $CDH \subset ABCDH$ quindi segue l'uscita dal ciclo con risultato:

$$AB^+ = ABCDH$$

5.3.1 Identificare le chiavi di uno schema

Per identificare le chiavi di uno schema R su cui è definito un insieme di dipendenze funzionali F , si utilizza il calcolo della chiusura di un insieme di attributi. Se la chiusura di X contiene R , abbiamo identificato una superchiave ma occorre verificare che i suoi sottoinsiemi non determinino tutto R .

Osservazione 5.3.1 – Identificare le chiavi di uno schema

Per identificare le chiavi di uno schema conviene partire dagli insiemi di attributi con cardinalità maggiore perché, se la loro chiusura non contiene R , è inutile calcolare la chiusura dei loro rispettivi sottoinsiemi. Inoltre, gli attributi che non compaiono a destra di nessuna dipendenza funzionale in F dovranno essere sicuramente in ogni chiave.

Esempio 5.3.2 – Identificare le chiavi di uno schema

Dato il seguente schema di relazione $R = (A, B, C, D, E, G, H)$ e il seguente insieme di dipendenze funzionali $F = \{AB \rightarrow D, G \rightarrow A, G \rightarrow B, H \rightarrow E, H \rightarrow G, D \rightarrow H\}$ determinare le 4 chiavi di R . Possiamo fin da subito notare che AB , G , D e H sono buoni candidati dato che E non determina nessun attributo, ma è determinato da H ; C non determina nessun attributo ma non è neppure determinato da nessun attributo, quindi sarà in ogni chiave. Date queste premesse possiamo calcolare le chiusure dei 4 candidati (controllando che i loro sottoinsiemi non determinano R):

$$\begin{aligned} ABC^+ &= ABCDEGH = R & AC^+ &= AC & BC^+ &= BC \\ GC^+ &= ABCDEGH = R \\ DC^+ &= ABCDEGH = R \\ HC^+ &= ABCDEGH = R \end{aligned}$$

Per le chiusure GC^+ , DC^+ , HC^+ è inutile controllare i loro sottoinsiemi dato che C è deve essere in ogni chiave e G , D e H , presi singolarmente, non determinano R .

Esempio 5.3.3 – Identificare le chiavi di uno schema

Dato il seguente schema di relazione $R = (A, B, C, D, E)$ e il seguente insieme di dipendenze funzionali $F = \{AB \rightarrow C, AC \rightarrow B, D \rightarrow E\}$ determinare le chiusure di $ABDE$, $ACDE$, $ABCD$ e dire se sono tutte e tre delle chiavi.

Calcolando le chiusure degli insiemi di attributi otteniamo:

$$ABDE^+ = ABDEC = R$$

$$ACDE^+ = ACDEB = R$$

$$ABCD^+ = ABCDE = R$$

In realtà $ABDE$ e $ABCD$ non sono chiavi perché $AB^+ = ABC$, per determinare R mancano D ed E , ma aggiungendo D determiniamo anche E (dalla dipendenza $D \rightarrow E$); quindi $ABD^+ = ABCDE = R$ è una chiave. Anche $ACDE$ non è una chiave, perché da $(AC)^+ = ABC$ mancano D ed E , ma aggiungendo D determiniamo anche E (dalla dipendenza $D \rightarrow E$); quindi $ACD^+ = ABCDE = R$ è una chiave.

Una volta individuate le chiavi di uno schema di relazione, possiamo determinare se lo schema è in 3NF.

Esempio 5.3.4 – Chiavi di uno schema e 3NF

Dato il seguente schema di relazione $R = (A, B, C, D, E, G, H)$ e il seguente insieme di dipendenze funzionali $F = \{AB \rightarrow CD, EH \rightarrow D, D \rightarrow H\}$ determinare una chiave di R , e, sapendo che la chiave è unica, verificare che R non è in 3NF.

Analizzando le dipendenze, cominciamo col considerare gli insiemi di attributi che compaiono a sinistra delle dipendenze stesse e calcoliamone le chiusure utilizzando l'algoritmo studiato:

$$AB^+ = \{A, B, C, D, H\}$$

AB non è superchiave

$$EH^+ = \{E, H, D\}$$

EH non è superchiave

$$D^+ = \{D, H\}$$

D non è superchiave

A questo punto osserviamo che G non compare mai nelle dipendenze, quindi dovrà far parte sicuramente della chiave dello schema. Inoltre l'attributo E non compare mai a destra delle dipendenze, quindi anche questo attributo non può essere determinato dalla chiave a meno di non farne parte. Osservando che alla chiusura di AB mancano appunto questi due attributi, proviamo a calcolare la chiusura dell'insieme $ABEG$:

$$ABEG^+ = \{A, B, E, G, C, D, H\}$$

Per verificare la minimalità della superchiave trovata, dobbiamo controllare che la chiusura di nessun sottoinsieme di $ABEG$ contenga tutti gli attributi dello schema. In teoria dovremmo calcolare le chiusure di tutti i possibili sottoinsiemi di $ABEG$, in pratica, tenendo conto del fatto che è inutile verificare la chiusura di sottoinsiemi che non contengono G , possiamo calcolare solo le chiusure di ABG , BEG , AEG :

$$ABG^+ = \{A, B, G, C, D, H\}$$

ABG non è superchiave

$$BEG^+ = \{B, E, G\}$$

BEG non è superchiave

$$AEG^+ = \{A, E, G\}$$

AEG non è superchiave

Abbiamo quindi verificato entrambe le condizioni che ci permettono di affermare che $ABEG$ è chiave dello schema dato.

Trovata la chiave $ABEG$, è sufficiente osservare la presenza anche della sola dipendenza $AB \rightarrow CD$ per verificare che R non è in 3NF. Infatti è una dipendenza parziale (la parte sinistra è contenuta propriamente in una chiave), e scomponendola in $AB \rightarrow C$ e $AB \rightarrow D$, in entrambi i casi l'attributo a destra non è primo e AB non è superchiave).

5.4 Decomposizione di uno schema

Osservazione 5.4.1 – La 3NF non basta

Uno schema che non è in 3NF può essere decomposto in un insieme di schemi in 3NF. Ad esempio lo schema $R = ABC$ con l'insieme di dipendenze funzionali $F = \{A \rightarrow B, B \rightarrow C\}$ non è in 3NF per la presenza in F^+ della dipendenza transitiva $B \rightarrow C$, dato che la chiave è evidentemente A . Quindi R può essere decomposto in:

$$R_1 = AB \text{ con } \{A \rightarrow B\} \quad R_2 = BC \text{ con } \{B \rightarrow C\}$$

oppure

$$R_1 = AB \text{ con } \{A \rightarrow B\} \quad R_2 = AC \text{ con } \{A \rightarrow C\}$$

Entrambi gli schemi sono in 3NF, tuttavia la seconda soluzione non è soddisfacente perché non soddisfa la dipendenza funzionale $B \rightarrow C$.

Consideriamo ora lo schema $R = ABC$ con l'insieme di dipendenze funzionali $F = \{A \rightarrow B, C \rightarrow B\}$. Lo schema non è in 3NF per la presenza in F^+ delle dipendenze parziali $A \rightarrow B$ e $C \rightarrow B$, dato che la chiave è AC tale schema può essere decomposto in:

$$R_1 = AB \text{ con } \{A \rightarrow B\} \quad R_2 = BC \text{ con } \{C \rightarrow B\}.$$

Tale schema pur preservando tutte le dipendenze in F^+ non è soddisfacente perché nel momento in cui si effettua un join per unire le istanze si avrà una perdita di informazioni.

In conclusione, quando si decompone uno schema per ottenerne uno in 3NF occorre tenere presenti altri due requisiti dello schema decomposto:

- deve preservare le dipendenze funzionali che valgono su ogni istanza legale dello schema originario;
- deve permettere di ricostruire, mediante join naturale, ogni istanza legale dello schema originario (senza aggiunta di informazione estranea).

Definizione 5.4.1 – Decomposizione di uno schema di relazione

Sia R uno schema di relazione. Una decomposizione di R è una famiglia $\rho = \{R_1, R_2, \dots, R_k\}$ di sottoinsiemi di R che ricopre R , ossia tali che:

$$R = \bigcup_{i=1}^k R_i$$

In altre parole, se lo schema R è composto da un certo insieme di attributi, decomporlo significa definire dei sottoschemi che contengono ognuno un sottoinsieme degli attributi di R . I sottoschemi possono avere attributi in comune, e la loro unione deve necessariamente contenere tutti gli attributi di R .

5.4.1 Preservare le dipendenze

Definizione 5.4.2 – Equivalenza tra insiemi di dipendenze

Sia R uno schema e siano F e G due insiemi di dipendenze funzionali su R . Tali insiemi vengono detti equivalenti, indicato come $F \equiv G$, se $F^+ = G^+$.

Lemma 5.4.1 – Equivalenza tra insiemi di dipendenze

Siano F e G due insiemi di dipendenze funzionali. Se $F \subseteq G^+$ allora $F^+ \subseteq G^+$.

Dimostrazione. Sia $f \in F^+ - F$ (è una dipendenza in F^+ che non compare in F). Ogni dipen-

denza funzionale in F è derivabile da G mediante gli assiomi di Armstrong (per l'ipotesi F si trova in G^+ e il Teorema 4.3.2 che abbiamo dimostrato afferma che $G^+ = G^A$). Sempre per il Teorema 4.3.2 che dimostra che $F^+ = F^A$ si ha che f in F^+ è derivabile dalle dipendenze in F mediante gli assiomi di Armstrong (F^A).

Definizione 5.4.3 – Decomposizioni che preservano le dipendenze

Sia R uno schema di relazione, F un insieme di dipendenze funzionali su R e $\rho = \{R_1, R_2, \dots, R_k\}$ una decomposizione di R . Diciamo che ρ preserva F se:

$$F \equiv \bigcup_{i=1}^k \pi_{R_i}(F)$$

dove $\pi_{R_i}(F) = \{X \rightarrow Y \mid X \rightarrow Y \in F^+ \wedge XY \subseteq R_i\}$ viene detta proiezione di F su R_i , ossia l'insieme di tutte le dipendenze in F^+ tali che il determinante e il determinato sono entrambi sotto insiemi di R_i .

Verificare se una decomposizione preserva un insieme di dipendenze funzionali F richiede che venga verificata l'equivalenza dei due insiemi di dipendenze funzionali F e $G = \bigcup_{i=1}^k \pi_{R_i}(F)$ e quindi la doppia inclusione $G^+ \subseteq F^+$ e che $F^+ \subseteq G^+$:

1. Per come è stato definito G , in questo caso, sarà sicuramente $G \subseteq F^+$. Infatti $G = \bigcup_{i=1}^k \pi_{R_i}(F)$, dove $\pi_{R_i}(F) = \{X \rightarrow Y \mid X \rightarrow Y \in F^+ \wedge XY \subseteq R_i\}$. Ogni proiezione di F che viene inclusa per definizione in G è un sottoinsieme di F^+ , quindi F^+ contiene G e per il Lemma 5.4.1 sulle chiusure questo implica che $G^+ \subseteq F^+$.
2. La verifica che $F \subseteq G^+$ (che poi implica che $F^+ \subseteq G^+$) può essere fatta con l'algoritmo che segue.

Algoritmo 5.4.1 – $F \subseteq G^+$

Dato uno schema R e due insiemi F e G di dipendenze funzionali su R , il seguente algoritmo determina se $F \subseteq G^+$:

```
begin
  successo:=true
  for every  $X \rightarrow Y \in F$ 
    calcola  $X_G^+$ 
    if  $Y \not\subseteq X_G^+$ 
      successo:=false
    end
```

end

E' importante calcolare X_G^+ perché per il Lemma 4.3.1 sappiamo che $X \rightarrow Y \in G^+ \iff Y \subseteq X_G^+$.

Per applicare tale algoritmo durante la verifica della preservazione di F , è necessario prima calcolare F^+ , in modo da poter calcolare G e successivamente ogni X_G^+ richiesto per verificare che $F \subseteq G^+$. Tuttavia, siccome il calcolo di F^+ richiede tempo esponenziale, è necessario calcolare i vari X_G^+ tramite un metodo alternativo.

Algoritmo 5.4.2 – Calcolo di X_G^+ a partire da F

Dato uno schema R con decomposizione $\rho = \{R_1, \dots, R_k\}$, dato un insieme F di dipendenze funzionali su R e posto $X \subseteq R$, con $G := \bigcup_{i=1}^k \pi_{R_i}(F)$, il seguente algoritmo calcola X_G^+ a partire da F :

```
begin
  Z := X
  S :=  $\emptyset$ 
```

```

for i := 1 to k
  S := S ∪ (Z ∩ Ri)F+ ∩ Ri
while S ⊈ Z
  Z := Z ∪ S
  for i := 1 to k
    S := S ∪ (Z ∩ Ri)F+ ∩ Ri
  XG+ := Z
end

```

Quindi stiamo raccogliendo gli attributi determinati funzionalmente dalla parte di Z che interseca R_i in base alle dipendenze in F^+ , e da questi selezioniamo quelli contenuti comunque in R_i (con l'intersezione $\cap R_i$). Partendo da un insieme X , stiamo ricostruendo la sua chiusura dalle proiezioni di F che compongono G , e da lì iterando (ciclo while) dall'insieme G^+ . Nel fare questo attraverso l'intersezione con R_i ci assicuriamo che la dipendenza sia valida nel singolo sottoschema.

Per verificare se $X \rightarrow Y$ è preservata, dobbiamo controllare se Y è contenuto nella copia finale della variabile Z .

Esempio 5.4.1 – Calcolo di X_G^+ a partire da F

Dato il seguente schema di relazione $R = (A, B, C, D)$ e il seguente insieme di dipendenze funzionali $F = \{AB \rightarrow C, D \rightarrow C, D \rightarrow B, C \rightarrow B, D \rightarrow A\}$ dire se la decomposizione $\rho = \{ABC, ABD\}$ preserva le dipendenze in F .

In base a quanto visto basta verificare che $F \subseteq G^+$ cioè che ogni dipendenza funzionale in F si trova in G^+ .

Ricordando la definizione di G :

$$G := \bigcup_{i=1}^k \pi_{R_i}(F) \quad \pi_{R_i}(F) = \{X \rightarrow Y \mid X \rightarrow Y \in F^+ \wedge XY \subseteq R_i\}$$

è inutile verificare le dipendenze $X \rightarrow Y$ la cui unione XY è contenuta in un sottoschema, perché per definizione fanno già parte di G . Quindi siccome $AB \rightarrow C$, $C \rightarrow B \in \pi_{ABC}(F) \subseteq G \subseteq G^+$, e $D \rightarrow B$, $D \rightarrow A \in \pi_{ABD}(F) \subseteq G \subseteq G^+$, non è necessario verificare tramite l'algoritmo che queste dipendenze vengono preservate.

Dunque l'unica dipendenza da verificare è $D \rightarrow C$. Applicando l'Algoritmo 2 per il calcolo di D_G^+ otteniamo $Z := D$ e $S = \emptyset$. Cicliamo sui due sottoschemi ABC e ABD :

$$S = S \cup (D \cap ABC)_F^+ \cap ABC = \emptyset \cup (\emptyset)_F^+ \cap ABC = \emptyset \cup \emptyset \cap ABC = \emptyset$$

$$S = S \cup (D \cap ABD)_F^+ \cap ABD = S \cup (D)_F^+ \cap ABD = \emptyset \cup DCBA \cap ABD = ABD$$

perché applicando l'Algoritmo 5.3 sulla chiusura di un insieme di attributi abbiamo $(D)_F^+ = DCBA$. Ora, essendo $ABD \not\subseteq D$ entriamo nel ciclo while, dove Z diventa $D \cup ABD = ABD$. Nel ciclo for all'interno del while cicliamo sui sottoschemi ABC e ABD e otteniamo:

$$S = S \cup (ABD \cap ABC)_F^+ \cap ABC = S \cup (AB)_F^+ \cap ABC = ABD \cup ABC \cap ABC = ABCD$$

$$S = S \cup (ABD \cap ABD)_F^+ \cap ABD = S \cup (ABD)_F^+ \cap ABD = ABCD \cup ABCD \cap ABD = ABCD \cup ABD = ABCD$$

perché applicando l'Algoritmo 5.3 sulla chiusura di un insieme di attributi abbiamo $(AB)_F^+ = ABC$ e $(ABD)_F^+ = ABCD$. Ora, $ABCD \not\subseteq ABD$, quindi rientriamo nel ciclo while con $Z = ABD \cup ABCD = ABCD$ e cicliamo sui sottoschemi ABC e ABD :

$$S = S \cup (ABCD \cap ABC)_F^+ \cap ABC = S \cup (ABC)_F^+ \cap ABC = ABCD \cup ABC \cap ABC = ABCD$$

$$S = S \cup (ABCD \cap ABD)_F^+ \cap ABD = S \cup (ABD)_F^+ \cap ABD = ABCD \cup ABCD \cap ABC = \\ ABCD \cup ABC = ABCD$$

Ora $ABCD \subseteq ABCD$ e l'algoritmo si ferma con $Z = (D)_G^+ = ABCD$ e poiché $C \in (D)_G^+$ la dipendenza è preservata.

Esempio 5.4.2 – Calcolo di X_G^+ a partire da F

Dato il seguente schema di relazione $R = (A, B, C, D, E)$ e il seguente insieme di dipendenze funzionali $F = \{AB \rightarrow E, B \rightarrow CE, ED \rightarrow C\}$ dire se la decomposizione $\rho = \{ABE, CDE\}$ preserva le dipendenze in F .

In base a quanto visto basta verificare che $F \subseteq G^+$ cioè che ogni dipendenza funzionale in F si trova in G^+ .

Dato che $AB \rightarrow E \in \pi_{ABE}(F) \subseteq G \subseteq G^+$, e $ED \rightarrow C \in \pi_{CDE}(F) \subseteq G \subseteq G^+$, non è necessario verificare tramite l'algoritmo che queste dipendenze vengono preservate.

Dunque l'unica dipendenza da verificare è $B \rightarrow CE$. Applicando l'Algoritmo 2 per il calcolo di D_G^+ otteniamo $Z := B$ e $S = \emptyset$. Cicliamo sui due sottoschemi ABE e CDE :

$$S = S \cup (B \cap ABE)_F^+ \cap ABE = \emptyset \cup (B)_A^+ \cap ABE = \emptyset \cup BCE \cap ABE = BE$$

$$S = BE \cup (B \cap CDE)_F^+ \cap CDE = BE \cup (\emptyset)_F^+ \cap CDE = BE$$

perché applicando l'Algoritmo 5.3 sulla chiusura di un insieme di attributi abbiamo $(B)_F^+ = BCE$. Ora, essendo $BE \not\subseteq B$ entriamo nel ciclo while, dove Z diventa $B \cup BE = BE$. Nel ciclo for all'interno del while cicliamo sui sottoschemi ABE e CDE e otteniamo:

$$S = BE \cup (BE \cap ABE)_F^+ \cap ABE = BE \cup (BE)_F^+ \cap ABE = BE \cup BCE \cap ABE = BE$$

$$S = BE \cup (BE \cap CDE)_F^+ \cap CDE = S \cup (E)_F^+ \cap CDE = BE \cup E \cap CDE = \\ BE \cup E = BE$$

perché applicando l'Algoritmo 5.3 sulla chiusura di un insieme di attributi abbiamo $(BE)_F^+ = BCE$ e $(E)_F^+ = E$. Ora, $BE \subseteq BE$ e l'algoritmo si ferma con $Z = (B)_G^+ = BE$ ma poiché $C \notin (B)_G^+$ la dipendenza non è preservata.

Teorema 5.4.1 – Correttezza dell'algoritmo per il calcolo di X_G^+ a partire da F

Sia R uno schema di relazione, F un insieme di dipendenze funzionali su R , $\rho = \{R_1, R_2, \dots, R_k\}$ una decomposizione di R e X un sottoinsieme di R . L'algoritmo per il calcolo di X_G^+ a partire da F calcola correttamente X_G^+ , dove $G = \bigcup_{i=1}^k \pi_{R_i}(F)$.

Dimostrazione. Indichiamo con Z_i il valore di Z dopo l' i -esima esecuzione, è facile vedere che $Z_i \subseteq Z_{i+1}$. Sia Z_f il valore di Z quando l'algoritmo termina, proveremo che:

$$A \in Z_f \iff A \in X_G^+$$

Mostriamo per induzione che $Z_i \subseteq X_G^+$:

Per il caso base si ha $Z_0 = X \subseteq X_G^+$;

Per l'ipotesi induttiva si ha per ogni $i \in \mathbb{N}$ che $Z_{i-1} \subseteq X_G^+$;

Per il passo induttivo, dato $A \in Z_i - Z_{i-1}$ deve esistere un indice j tale che $A \in (Z_{i-1} \cap R_j)_F^+ \cap R_j$ (perché A è stato aggiunto durante nell' i -esima iterazione e non era in Z_{i-1}).

Poiché $A \in (Z_{i-1} \cap R_j)_F^+$, per il Lemma 4.3.1, si ha $(Z_{i-1} \cap R_j) \rightarrow A \in F^+$. Poiché $(Z_{i-1} \cap R_j) \rightarrow A \in F^+$, $A \in R_j$ e $Z_{i-1} \cap R_j \subseteq R_j$, si ha, per la definizione di G , che $(Z_{i-1} \cap R_j) \rightarrow A \in G$. Per l'ipotesi induttiva (con il Lemma 4.3.1) si ha $X \rightarrow Z_{i-1} \in G^+$, per la regola della decomposizione si ha anche che $X \rightarrow (Z_{i-1} \cap R_j) \in G^+$ e, quindi,

per l'assioma della transitività $(X \rightarrow (Z_{i-1} \cap R_j) \in G^+, (Z_{i-1} \cap R_j) \rightarrow A \in G^+)$, che $X \rightarrow A \in G^+$, cioè $A \in X_G^+$. Quindi $Z_i \subseteq X_G^+$.

5.4.2 Join senza perdita

Abbiamo sottolineato varie volte che una buona decomposizione deve soddisfare 3 condizioni: ogni sottoschema deve essere in 3NF; la decomposizione deve preservare tutte le dipendenze in F^+ ; la decomposizione deve permettere di ricostruire una istanza legale decomposta senza perdita di informazione (join senza perdita).

Esempio 5.4.3 – Join con perdita

Consideriamo lo schema $R = ABC$ con l'insieme di dipendenze funzionali $F = \{A \rightarrow B, C \rightarrow B\}$, lo schema non è in 3NF per la presenza in F^+ delle dipendenze parziali $A \rightarrow B$ e $C \rightarrow B$, dato che la chiave è AC . Tale schema può essere decomposto in:

$$R_1 = AB \text{ con } \{A \rightarrow B\} \quad R_2 = BC \text{ con } \{C \rightarrow B\}$$

Tale decomposizione pur preservando tutte le dipendenze in F^+ non è soddisfacente. Infatti, considerando l'istanza legale R , in base alla decomposizione data, si divide in R_1 e R_2 :

R			R ₁		R ₂	
A	B	C	A	B	B	C
a1	b1	c1	a1	b1	b1	c1
a2	b1	c2	a2	b1	b1	c2

Ora dovrebbe essere possibile ricostruire R , invece se si effettua il join delle due istanze legali risultanti dalla decomposizione si ottiene:

R		
A	B	C
a1	b1	c1
a2	b1	c2
a1	b1	c2
a2	b1	c1

con tuple estranee alla realtà di interesse quindi con perdita di informazione.

Definizione 5.4.4 – Join senza perdita

Sia R uno schema di relazione. Una decomposizione $\rho = \{R_1, R_2, \dots, R_k\}$ di R ha un join senza perdita se per ogni istanza legale r di R si ha:

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$$

Teorema 5.4.2

Sia R uno schema di relazione e $\rho = \{R_1, R_2, \dots, R_k\}$ una decomposizione di R . Per ogni istanza legale r di R , indicato con $m_\rho(r) = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$, si ha:

- (a) $r \subseteq m_\rho(r)$
- (b) $\pi_{R_i}(m_\rho(r)) = \pi_{R_i}(r)$
- (c) $m_\rho(m_\rho(r)) = m_\rho(r)$

Dimostrazione.

- (a) Data una qualsiasi tupla $t \in r$, si ha che:

$$t \in r \implies t \in \{t[R_1]\} \bowtie \dots \{t[R_k]\} \subseteq \pi_{R_1}(r) \bowtie \dots \pi_{R_k}(r) = m_\rho(r)$$

- (b) Poiché $r \subseteq m_\rho(r)$, effettuando una proiezione con $R_i \in \rho$ su entrambe, ne segue che $\pi_{R_i}(r) \subseteq \pi_{R_i}(m_\rho(r))$. Inoltre, per definizione di proiezione, si ha che:

$$t_{R_i} \in \pi_{R_i}(m_\rho(r)) \implies \exists t' \in m_\rho(r) | t_{R_i} = t'[R_i]$$

Infine, per definizione stessa di $m_\rho(r)$, si ha che:

$$t' \in m_\rho(r) \implies \exists t_1, \dots, t_k \in r | \forall R_j \in \rho, t_j[R_j] = t'[R_j]$$

In particolare, quindi, otteniamo che:

$$t_{R_i} \in \pi_{R_i}(m_\rho(r)) \implies t_{R_i} = t'[R_i] = t_i[R_i] \in \pi_{R_i}(r) \implies \pi_{R_i}(m_\rho(r)) \subseteq \pi_{R_i}(r)$$

- (c) Siccome $\pi_{R_i}(r) = \pi_{R_i}(m_\rho(r))$, allora si ha che:

$$m_\rho(m_\rho(r)) = \pi_{R_1}(m_\rho(r)) \bowtie \dots \bowtie \pi_{R_k}(m_\rho(r)) = \pi_{R_1}(r) \bowtie \dots \bowtie \pi_{R_k}(r) = m_\rho(r)$$

Per verificare se la decomposizione data produce un join senza perdita utilizziamo un algoritmo di verifica in tempo polinomiale.

Algoritmo 5.4.3 – Verifica del join senza perdita in una decomposizione

Dato uno schema di relazione R , un insieme F di dipendenze funzionali su R , una decomposizione $\rho = \{R_1, R_2, \dots, R_k\}$ di R , il seguente algoritmo determina se ρ presenta un join senza perdita.

Si costruisce una tabella r con R colonne e ρ righe in modo che l'incrocio dell' i -esima riga e della j -esima colonna ha:

$$r_{i,j} = \begin{cases} "a_j" & \text{se } A_j \in R_i \\ "b_{ij}" & \text{se } A_j \notin R_i \end{cases}$$

e si applica il seguente pseudocodice:

```

begin
  repeat
    for every  $X \rightarrow Y \in F$ 
      if  $t_1[X] = t_2[X]$  and  $t_1[Y] \neq t_2[Y]$ 
        for every attribute  $A_j \in Y$ 
          if  $t_1[A_j] = "a_j"$ 
             $t_2[A_j] := t_1[A_j]$ 
          else
             $t_1[A_j] := t_2[A_j]$ 
    until  $r$  ha una riga con tutte "a" or  $r$  non è cambiato
    if  $r$  ha una riga con tutte "a"
       $\rho$  ha un join senza perdita
    else
       $\rho$  non ha un join senza perdita
  end

```

Per i nostri scopi, trasformiamo l'istanza iniziale in una istanza legale e per fare questo, dobbiamo fare in modo che tutte le dipendenze in F siano soddisfatte. Allora, ogni volta che troviamo due tuple uguali sugli attributi a sinistra, facciamo in modo che siano uguali anche su quelli a destra. Nel fare questo, diamo la precedenza alle a . L'algoritmo di ferma quando tutte le coppie di tuple soddisferanno la condizione che se gli attributi delle parti sinistre delle dipendenze sono uguali, lo sono anche gli attributi delle parti destre, ma questo significa che alla fine r è stata trasformata in un'istanza legale di R .

Esempio 5.4.4 – Verifica del join senza perdita in una decomposizione

Dato lo schema $R = ABCDE$ con decomposizione $\rho = \{AC, ADE, CDE, AD, B\}$ e con insieme di dipendenze $F = \{C \rightarrow D, AB \rightarrow E, D \rightarrow B\}$, vogliamo verificare se ρ presenti un join senza perdita. Partiamo costruendo l'istanza di r secondo le regole date:

	A	B	C	D	E
AC	a1	b12	a3	b14	b15
ADE	a1	b22	b23	a4	a5
CDE	b31	b32	a3	a4	a5
AD	a1	b42	b43	a4	b45
B	b51	a2	b53	b54	b55

Effettuiamo quindi la prima iterazione del ciclo: considerando $C \rightarrow D \in F$, notiamo che la prima e la terza tupla sono uguali nel determinante C , dunque modifichiamo $b14 \rightarrow a4$ affinché esse siano uguali anche nel determinato D ; considerando $AB \rightarrow E \in F$, notiamo che tale dipendenza è già soddisfatta; considerando $D \rightarrow B \in F$, notiamo che la prima (poiché abbiamo già modificato $b14 \rightarrow a4$), la seconda, la terza e la quarta tupla sono uguali nel determinante D , dunque modifichiamo $b22 \rightarrow b12$, $b32 \rightarrow b12$ e $b42 \rightarrow b12$ affinché esse siano uguali anche nel determinato B .

	A	B	C	D	E
AC	a1	b12	a3	b14→a4	b15
ADE	a1	b22→b12	b23	a4	a5
CDE	b31	b32→b12	a3	a4	a5
AD	a1	b42→b12	b43	a4	b45
B	b51	a2	b53	b54	b55

Siccome la tabella è stata modificata, effettuiamo un'altra iterazione del ciclo: considerando $C \rightarrow D \in F$, notiamo che tale dipendenza è già soddisfatta; considerando $AB \rightarrow E \in F$, notiamo la prima, la seconda e la quarta tupla sono uguali nel determinante AB , dunque modifichiamo $b15 \rightarrow a5$ e $b45 \rightarrow a5$ in modo che siano uguali nel determinato E ; considerando $D \rightarrow B \in F$, notiamo che tale dipendenza è già soddisfatta.

	A	B	C	D	E
AC	a1	b12	a3	a4	b15→a5
ADE	a1	b12	b23	a4	a5
CDE	b31	b12	a3	a4	a5
AD	a1	b12	b43	a4	b45→a5
B	b51	a2	b53	b54	b55

Siccome la tabella è stata modificata, allora effettuiamo un'altra iterazione del ciclo: considerando $C \rightarrow D \in F$, notiamo che tale dipendenza è già soddisfatta; considerando $AB \rightarrow E \in F$, notiamo che tale dipendenza è già soddisfatta; considerando $D \rightarrow B \in F$, notiamo che tale dipendenza è già soddisfatta.

	A	B	C	D	E
AC	a1	b12	a3	a4	a5
ADE	a1	b12	b23	a4	a5
CDE	b31	b12	a3	a4	a5
AD	a1	b12	b43	a4	a5
B	b51	a2	b53	b54	b55

Siccome la tabella non è stata modificata, allora l'algoritmo termina stabilendo che ρ non presenta un join senza perdita, poiché non esiste alcuna riga contenente tutti valori "a".

Teorema 5.4.3 – Correttezza dell'algoritmo per la verifica del join

Sia R uno schema di relazione, F un insieme di dipendenze funzionali su R e $\rho = \{R_1, R_2, \dots, R_k\}$ una decomposizione di R . L'Algoritmo determina che ρ presenta un join senza perdita se e solo se esiste una tupla in r che contiene tutte "a" una volta terminato.

Dimostrazione. Supponiamo per assurdo che ρ abbia un join senza perdita ($m_\rho(r) = r$) e che quando l'algoritmo termina la tabella r non abbia una tupla con tutte "a". La tabella r può essere interpretata come un'istanza legale di R in quanto l'algoritmo termina quando non ci sono più violazioni delle dipendenze in F . Poiché nessun simbolo "a" che compare nella tabella costruita inizialmente viene mai modificato dall'algoritmo, per ogni $i, i = 1, \dots, k$, $\pi_{R_i}(r)$ contiene una tupla con tutte "a" (infatti se si proietta l'istanza r sugli attributi di R_i , e precisamente nella riga corrispondente al sottoschema R_i , si nota come almeno una riga contiene tutte le "a") pertanto $m_\rho(r)$ contiene sicuramente una tupla con tutte "a" e, quindi, $m_\rho(r) \neq r$ (contraddizione).

5.4.3 Copertura minimale**Definizione 5.4.5 – Copertura minimale**

Sia F un insieme di dipendenze funzionali. Una copertura minimale di F è un insieme G di dipendenze funzionali equivalente ad F tale che:

- per ogni dipendenza funzionale in G la parte destra è un singleton, cioè è costituita da un unico attributo (ogni attributo nella parte destra è non ridondante);
- per nessuna dipendenza funzionale $X \rightarrow A$ in G esiste $X' \subset X$ tale che $G \equiv G - \{X \rightarrow A\} \cup \{X' \rightarrow A\}$ (ogni attributo nella parte sinistra è non ridondante);
- per nessuna dipendenza funzionale $X \rightarrow A$ in G , $G \equiv G - \{X \rightarrow A\}$ (ogni dipendenza è non ridondante).

Algoritmo 5.4.4 – Calcolare una copertura minimale

Per ogni insieme di dipendenze funzionali F esiste una copertura minimale equivalente ad F che può essere ottenuta in tempo polinomiale in tre passi:

1. usando la regola della decomposizione, le parti destre delle dipendenze funzionali vengono ridotte a singleton;
2. ogni dipendenza funzionale $A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A$ in F tale che

$$F \equiv F - \{A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A\} \cup \{A_1 A_2 \dots A_{i-1} A_{i+1} \dots A_n \rightarrow A\}$$

viene sostituita con $A_1 A_2 \dots A_{i-1} A_{i+1} \dots A_n \rightarrow A$ (o eliminata se quest'ultima appartiene già a F). Questo processo viene ripetuto finché possibile, cioè quando tutti gli attributi delle parti sinistre delle dipendenze funzionali risultano non ridondanti;

3. ogni dipendenza funzionale $X \rightarrow A$ in F tale che $F \equiv F - \{X \rightarrow A\}$ viene eliminata, in quanto risulta ridondante.

Osservazione 5.4.2 – Semplificazione del passo 2

Nel passo 2, ogni volta che vogliamo verificare la ridondanza di un attributo nella parte di una dipendenza, assumiamo di indicare con F l'insieme che contiene la dipendenza originaria $A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A$ e con G l'insieme che contiene al suo posto la dipendenza $A_1 A_2 \dots A_{i-1} A_{i+1} \dots A_n \rightarrow A$. Notiamo prima di tutto che i due insiemi differiscono esattamente in una dipendenza (A_i), le altre sono uguali, e quindi banalmente appartengono alla chiusura

di entrambi gli insiemi. Per stabilire quindi l'equivalenza resta da verificare che:

$$A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A \in G^+ \wedge A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A \in F^+$$

Tuttavia, non è necessario verificare se $A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A \in G^+$, poiché $A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \subset A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n$ e per riflessività avremo che $A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n$ e per transitività con $A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A$ otteniamo $A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A$.

Resta da verificare se $A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n \rightarrow A \in F^+$ e per fare questo usiamo l'Algoritmo 5.3 per verificare se $A \in (A_1 A_2 \dots A_{i-1} A_i A_{i+1} \dots A_n)_F^+$.

Osservazione 5.4.3 – Semplificazione passo 2

Nelle verifiche successive, relative alla riduzione di ulteriori dipendenze nel passo 2, è inutile ricalcolare le chiusure per attributi o gruppi di attributi per i quali tale calcolo sia già stato effettuato.

Dato per esempio X , siccome $(X)_F^+ = \{A | X \rightarrow A \in F^+\}$, e siccome il verso della verifica porta a calcolare le chiusure rispetto ad un insieme di dipendenze che è F iniziale oppure un insieme G che è stato già dimostrato equivalente ad F , se $X \rightarrow A \in F^+$ allora vale anche $X \rightarrow A \in G^+$, cioè $(X)_F^+ = (X)_G^+$.

Osservazione 5.4.4 – Semplificazione passo 3

Assumiamo di indicare con F l'insieme che contiene la dipendenza $X \rightarrow A$, e con G l'insieme in cui tale dipendenza è stata eliminata. Anche in questo caso, i due insiemi differiscono per una sola dipendenza, anzi si verifica che $G \subseteq F$, quindi sappiamo già che $G^+ \subseteq F^+$. Resta quindi da verificare che sia $F^+ \subseteq G^+$, che sarà sicuramente vero se $F \subseteq G^+$ (per il Lemma 5.4.1). In particolare, basta verificare se $X \rightarrow A \in G^+$ e cioè se $A \in (X)_G^+$.

In questo caso però le chiusure di attributi o gruppi di attributi vanno ricalcolate, perché il verso della verifica porta a calcolare le chiusure rispetto ad un insieme di dipendenze che non è stato ancora dimostrato essere equivalente ad F .

Notiamo inoltre che se $X \rightarrow A \in F$ ma non esiste $Y \rightarrow A \in F$ con $Y \neq X$, allora è inutile provare ad eliminare $X \rightarrow A$, in quanto eliminando tale dipendenza non saremmo più in grado di determinare funzionalmente A .

Possono esserci più coperture minimali per un dato insieme di dipendenze funzionali. Usando l'algoritmo si può sempre trovare almeno una copertura minimale per qualunque insieme F di dipendenze. Può anche succedere che F sia già in forma minimale, e quindi non ci sia da fare alcuna riduzione.

Esempio 5.4.5 – Copertura minimale

Troviamo la copertura minimale del seguente insieme di dipendenze funzionali $F = \{AB \rightarrow CD, C \rightarrow E, AB \rightarrow E, ABC \rightarrow D\}$.

1. Prima di tutto, scomponiamo le dipendenze con la regola della decomposizione, ottenendo che:

$$F = \{AB \rightarrow C, AB \rightarrow D, C \rightarrow E, AB \rightarrow E, ABC \rightarrow D\}$$

2. Ora dobbiamo verificare se nelle dipendenze ci sono ridondanze nelle parti sinistre.

Consideriamo quindi la dipendenza $AB \rightarrow C \in F$ e verifichiamo se $A \rightarrow C, B \rightarrow C \in F^+$, dunque se $C \in A_F^+$ e se $C \in B_F^+$: $A_F^+ = A \implies C \notin A_F^+$; $B_F^+ = B \implies C \notin B_F^+$ dunque, non possiamo rimpiazzare la dipendenza $AB \rightarrow C$.

Procedendo analogamente, verifichiamo che $A \rightarrow E, B \rightarrow E, A \rightarrow D, B \rightarrow D \notin F^+$, dunque ne traiamo che $AB \rightarrow E, AB \rightarrow D \in F$ non possano essere rimpiazzate.

Infine, considerando $ABC \rightarrow D \in F$, sappiamo già che $AB \rightarrow D \in F \subseteq F^+$, dunque tale dipendenza può essere rimpiazzata data $AB \rightarrow D$ (e di conseguenza rimossa, poiché già presente in F)

Al termine dello step 2, quindi, abbiamo che:

$$F = \{AB \rightarrow C, AB \rightarrow D, C \rightarrow E, AB \rightarrow E\}$$

3. Vediamo ora se questo insieme contiene delle dipendenze ridondanti.

C e D sono determinati rispettivamente solo da $AB \rightarrow C$ e $AB \rightarrow D$, dunque tali dipendenze non possono essere rimosse.

Considerando $C \rightarrow E \in F$, verifichiamo se $E \in C_{F-\{C \rightarrow E\}}^+$ affinché $C \rightarrow E \in F - \{C \rightarrow E\}^+$. Siccome $E \notin C_{F-\{C \rightarrow E\}}^+ = C$, ne segue che $C \rightarrow E$ non possa essere scartata.

Considerando $AB \rightarrow E \in F$, verifichiamo se $E \in AB_{F-\{AB \rightarrow E\}}^+$ affinché $AB \rightarrow E \in F - \{AB \rightarrow E\}^+$. Siccome $E \in AB_{F-\{AB \rightarrow E\}}^+ = ABCDE$, ne segue che $AB \rightarrow E$ possa essere scartata.

Infine, otteniamo che:

$$F = \{AB \rightarrow C, AB \rightarrow D, C \rightarrow E\}$$

è la copertura minimale di F .

Esempio 5.4.6 – Copertura minimale

Troviamo la copertura minimale del seguente insieme di dipendenze funzionali $F = \{BC \rightarrow DE, C \rightarrow D, B \rightarrow D, E \rightarrow L, D \rightarrow A, BC \rightarrow AL\}$

1. Prima di tutto, decomponiamo le dipendenze:

$$F = \{BC \rightarrow D, BC \rightarrow E, C \rightarrow D, B \rightarrow D, E \rightarrow L, D \rightarrow A, BC \rightarrow A, BC \rightarrow L\}$$

2. Siccome BC è l'unico determinante su cui potremmo applicare lo step due, calcoliamo $B_F^+ = ABD$ e $C_F^+ = ACD$, da cui otteniamo che:

$BC \rightarrow D$ può essere scartata direttamente, poiché sia $B \rightarrow D$ sia $C \rightarrow D$ sono entrambe già in F .

$E \notin B_F^+ \implies B \rightarrow E \notin F^+$ (e anche $E \notin C_F^+ \implies C \rightarrow E \in F^+$), dunque $BC \rightarrow E$ non può essere rimpiazzata.

$A \in B_F^+ \implies B \rightarrow A \in F^+$ e $A \in C_F^+ \implies C \rightarrow A \in F^+$, dunque $BC \rightarrow A$ può essere rimpiazzata sia da $B \rightarrow A$ sia da $C \rightarrow A$ (dunque, in base alla scelta, otterremo due coperture minimali diverse). Nel nostro caso, sceglieremo $B \rightarrow A$.

Siccome $L \notin B_F^+ \implies B \rightarrow L \notin F^+$ e siccome $L \notin C_F^+ \implies C \rightarrow L \notin F^+$, la dipendenza $BC \rightarrow L$ non può essere rimpiazzata.

Al termine dello step 2, quindi, abbiamo che:

$$F = \{BC \rightarrow E, C \rightarrow D, B \rightarrow D, E \rightarrow L, D \rightarrow A, B \rightarrow A, BC \rightarrow L\}$$

3. Procediamo quindi con lo step 3:

Siccome E è determinato solo da $BC \rightarrow E$, allora non possiamo scartare tale dipendenza.

Considerando $C \rightarrow D$, si ha che $D \notin C_{F-\{C \rightarrow D\}}^+ = C \iff C \rightarrow D \notin F - \{C \rightarrow D\}^+$, dunque $C \rightarrow D$ non può essere scartata.

Considerando $B \rightarrow D$, si ha che $D \notin B_{F-\{B \rightarrow D\}}^+ = BA \iff B \rightarrow D \notin F - \{B \rightarrow D\}^+$, dunque $B \rightarrow D$ non può essere scartata.

Considerando $E \rightarrow L$, si ha che $L \notin E_{F-\{E \rightarrow L\}}^+ = E \iff E \rightarrow L \notin F - \{E \rightarrow L\}^+$, dunque $E \rightarrow L$ non può essere scartata.

Considerando $D \rightarrow A$, si ha che $A \notin D_{F-\{D \rightarrow A\}}^+ = D \iff D \rightarrow A \notin F - \{D \rightarrow A\}^+$, dunque $D \rightarrow A$ non può essere scartata.

Considerando $B \rightarrow A$, si ha che $A \in B_{F-\{B \rightarrow A\}}^+ = BDA \iff B \rightarrow A \in F - \{B \rightarrow A\}^+$, dunque $B \rightarrow A$ può essere scartata.

Considerando $BC \rightarrow L$, si ha che $L \in BC_{F-\{B \rightarrow A, BC \rightarrow L\}}^+ = BCEDLA \iff BC \rightarrow L \in F - \{B \rightarrow A, BC \rightarrow D\}^+$, dunque $BC \rightarrow L$ può essere scartata.

Infine, otteniamo che:

$$F = \{BC \rightarrow E, C \rightarrow D, B \rightarrow D, E \rightarrow L, D \rightarrow A\}$$

è la copertura minimale.

5.4.4 Algoritmo di decomposizione

Algoritmo 5.4.5 – Algoritmo di decomposizione

Dato uno schema R e un insieme F , che è una copertura minimale di dipendenze funzionali su R , il seguente algoritmo calcola in tempo polinomiale una decomposizione ρ di R tale che ogni sottoschema è in 3NF e ρ preserva F :

```

begin
  S := ∅
  for every A ∈ R | A non è coinvolto in nessuna dipendenza funzionale in F
    S := S ∪ {A}
  if S ≠ ∅
    R := R - S
    ρ := ρ ∪ {S}
  if esiste una dipendenza funzionale in F che coinvolge tutti gli attributi in R
    ρ := ρ ∪ {R}
  else
    for every X → A ∈ F
      ρ := ρ ∪ {XA}
end

```

Proposizione 5.4.1 – Ottenere una buona decomposizione

Sia R uno schema di relazione ed F un insieme di dipendenze funzionali su R , che è una copertura minimale. L'Algoritmo di decomposizione permette di calcolare in tempo polinomiale una decomposizione ρ di R tale che: (1) ρ preserva F ; (2) ogni schema di relazione in ρ è in 3NF.

Dimostrazione.

1. Sia $G = \bigcup_{i=1}^k \pi_{R_i}(F)$. Poiché per ogni dipendenza funzionale $X \rightarrow A \in F$ si ha che $XA \in \rho$ si ha che questa dipendenza di F sarà sicuramente in G , quindi $G \supseteq F$ e, quindi $G^+ \supseteq F^+$. L'inclusione $G^+ \subseteq F^+$ è banalmente verificata in quanto, per definizione, $G \subseteq F^+$.
2. Analizziamo i diversi casi che si possono presentare: se $S \in \rho$, ogni attributo in S fa parte della chiave e quindi, banalmente, S è in 3NF; se $R \in \rho$ esiste una dipendenza funzionale in F che coinvolge tutti gli attributi in R . Poiché F è una copertura minimale tale dipendenza avrà la forma $R - A \rightarrow A$; poiché F è una copertura minimale, non ci può essere una dipendenza funzionale $X \rightarrow A$ in F^+ tale che $X \subset R - A$ e, quindi, $R - A$ è chiave in R . Sia $Y \rightarrow B$ una qualsiasi dipendenza in F ; se $B = A$ allora, poiché F è una copertura minimale, $Y = R - A$ (cioè, Y è una superchiave); se $B \neq A$ allora $B \in R - A$ e quindi B è primo.

3. Se $XA \in \rho$, poiché F è una copertura minimale, non ci può essere una dipendenza funzionale $X' \rightarrow A$ in F^+ tale che $X' \subset X$ e, quindi, X è chiave in XA . Sia $Y \rightarrow B$ una qualsiasi dipendenza in F tale che $YB \subseteq XA$; se $B = A$ allora, poiché F è una copertura minimale, $Y = X$ (cioè, Y è una superchiave); se $B \neq A$ allora $B \in X$ e quindi B è primo.

Esempio 5.4.7 – Algoritmo di decomposizione

Consideriamo lo schema $R = ABCDEH$ e l'insieme di dipendenze $F = \{AB \rightarrow CD, C \rightarrow E, AB \rightarrow E, ABC \rightarrow D\}$.

Prima di tutto, verifichiamo l'unicità della chiave dello schema: troviamo prima l'intersezione

$$ABEH \cap ABCDH \cap ABCDH \cap ABCEH = ABH$$

e vediamo se la chiusura dell'intersezione coincide con R :

$$ABH^+ = ABCDEH = R$$

Poiché ABH è l'unica chiave di R , notiamo subito che tale schema non è in 3NF, poiché la dipendenza $C \rightarrow E$ viola la condizione richiesta in quanto C non è una superchiave ed E non è un primo. Cerchiamo quindi una copertura minimale di F , in modo da poter utilizzare l'algoritmo di decomposizione.

Decomponiamo tutte le dipendenze di F :

$$F = \{AB \rightarrow C, AB \rightarrow D, C \rightarrow E, AB \rightarrow E, ABC \rightarrow D\}$$

Cerchiamo i determinanti ridondanti:

$A \rightarrow C, B \rightarrow C \notin F^+$ poiché $C \notin A_F^+ = A$ e $C \notin B_F^+ = A$, dunque $AB \rightarrow C$ non può essere ridotta; $A \rightarrow D, B \rightarrow D \notin F^+$ poiché $D \notin A_F^+ = A$ e $D \notin B_F^+ = A$, dunque $AB \rightarrow D$ non può essere ridotta; $A \rightarrow E, B \rightarrow E \notin F^+$ poiché $E \notin A_F^+ = A$ e $E \notin B_F^+ = A$, dunque $AB \rightarrow E$ non può essere ridotta; $AB \rightarrow D, C \rightarrow D \in F \implies AB \rightarrow D, C \rightarrow D \in F^+$, dunque $ABC \rightarrow D$ può essere ridotta sia in $AB \rightarrow D$ sia in $C \rightarrow D$ e dunque scartata poiché entrambe sono già in F .

L'insieme di dipendenze senza determinanti ridondanti quindi sarà:

$$F = \{AB \rightarrow C, AB \rightarrow D, C \rightarrow E, AB \rightarrow E\}$$

Cerchiamo quindi le dipendenze ridondanti:

C e D sono determinati solo da $AB \rightarrow C$ e $AB \rightarrow D$, dunque non possono essere rimosse; Considerando $C \rightarrow E$, si ha che $E \notin D_{F-\{C \rightarrow E\}}^+ = C \iff C \rightarrow E \notin F - \{C \rightarrow E\}^+$, dunque $C \rightarrow E$ non può essere scartata; Considerando $AB \rightarrow E$, si ha che $E \in AB_{F-\{AB \rightarrow E\}}^+ = ABCDE \iff AB \rightarrow E \in F - \{AB \rightarrow E\}^+$, dunque $AB \rightarrow E$ può essere scartata.

La copertura minimale di F quindi sarà:

$$F = \{AB \rightarrow C, AB \rightarrow D, C \rightarrow E\}$$

A questo punto, possiamo applicare l'algoritmo di decomposizione:

L'attributo H non compare in alcuna dipendenza, dunque $S := \{H\}$. Siccome $S = \{H\} \neq \emptyset$, allora $R := R - S = ABCDEH - H = ABCDE$ e $\rho := \rho \cup S = \{H\}$. Siccome $\nexists X \rightarrow A \in F \mid X \cup A = R = ABCDE$, allora entriamo nel ciclo for, dunque $\rho := \rho \cup \{ABC, ABD, CE\} = \{H, ABC, ABD, CE\}$. La decomposizione ρ ottenuta preserva F e tutti i suoi sottoschemi sono in 3NF. Infine, affinché ρ sia una buona decomposizione, dunque presenti anche un join senza perdita, è sufficiente aggiungere un sottoschema composto dalla chiave ABH alla decomposizione ρ .

Dunque, una buona decomposizione di R con copertura minimale $F = \{AB \rightarrow C, AB \rightarrow D, C \rightarrow E\}$ risulta essere:

$$\rho = \{H, ABC, ABD, CE, ABH\}$$

Capitolo 6

Organizzazione fisica dei dati

Un requisito fondamentale di un sistema di gestione di basi di dati è l'efficienza, cioè la capacità di rispondere alle richieste dell'utente il più rapidamente possibile. Poiché una particolare organizzazione fisica dei dati può rendere efficiente l'elaborazione di particolari richieste, bisogna tener presente quali operazioni saranno effettuate più frequentemente.

Generalmente ad ogni relazione corrisponde un file di record, dove ogni attributo corrisponde ad un campo e ogni tupla ad un record.

I file sono memorizzati su disco, ogni disco è partizionato in blocchi di formato fisso e ogni blocco corrisponde generalmente ad un settore di una traccia. Il costo di un accesso a disco, cioè del trasferimento di un blocco da o a memoria secondaria è molto superiore a quello di elaborazione di un blocco in memoria principale. Pertanto la valutazione del tempo di risposta del sistema ad una richiesta dell'utente viene effettuata in termini di numero di accessi a disco.

In realtà non sempre quando si deve elaborare un blocco è necessario effettuare un accesso a disco, in quanto il sistema quando trasferisce un blocco in memoria principale lo memorizza in un buffer e mantiene i blocchi trasferiti fin quando è possibile ricordando quali blocchi sono nei buffer. Inoltre il costo di un accesso a disco non è fisso, ma dipende dalla posizione del blocco rispetto all'ultimo blocco trasferito in quanto da ciò dipendono sia lo spostamento della testina da un cilindro all'altro sia la rotazione del disco. Tuttavia per valutare i costi delle operazioni elementari su file con determinata organizzazione fisica assumeremo che ogni lettura e scrittura di un blocco richieda il trasferimento del blocco da o a memoria secondaria e che ogni accesso a memoria secondaria abbia costo costante.

6.1 Record

Come abbiamo detto i record contengono campi che corrispondono agli attributi di relazione. Oltre a questi campi, però, un record può contenere campi che forniscono informazioni sul record stesso o che puntano ad altri record. Ad esempio alcuni byte possono essere utilizzati per specificare il tipo del record, la sua lunghezza o un bit di cancellazione oppure, se si vuole spostare il record all'interno del blocco, si può utilizzare una coppia (b, k) , dove b è l'indirizzo del blocco che contiene il record e k il valore di uno o più campi che servono come chiave.

Per poter accedere ad un campo di un record contenente un dato è necessario sapere qual'è il primo byte del campo nel record. Se tutti i campi del record hanno lunghezza fissa, basta ordinarli, infatti, una volta ordinati, l'inizio di ciascun campo sarà sempre ad un numero fisso di byte dall'inizio del record (*offset del campo*). Se il record contiene campi a lunghezza variabile allora l'offset di un campo può variare da un record all'altro, in tal caso si possono usare due strategie: o all'inizio di ogni campo c'è un contatore che specifica la lunghezza del campo in numero di byte, oppure all'inizio del record ci sono dei puntatori all'inizio di ciascun campo a lunghezza variabile.

6.2 Blocchi

Analogamente a quanto accade per un record, anche in un blocco può essere riservato spazio per memorizzare informazioni sul blocco stesso o per collegarlo ad altri blocchi in una lista.

Se un blocco contiene solo record di lunghezza fissa allora il blocco può essere suddiviso in aree (sotto-blocchi) di lunghezza fissa ciascuna delle quali può contenere un record. Se un blocco contiene record di lunghezza variabile per accedere ai record si possono usare due strategie: o si pone in ogni record un campo che ne specifica la lunghezza e, per calcolare il byte di inizio di un record, si somma all'offset del record precedente la sua lunghezza, oppure si pone all'inizio del blocco una directory contenente i puntatori ai record nel blocco. Poiché il numero di record che possono entrare in un blocco è in questo caso variabile, la directory può essere realizzata in uno dei modi seguenti: preceduta da un campo che specifica quanti sono i puntatori, come una lista di puntatori che termina con il valore 0 oppure di dimensione fissa con valore 0 negli spazi che non contengono puntatori.

6.3 File

In questo paragrafo esamineremo diversi tipi di organizzazione fisica di file che consentono la ricerca di record in base al valore di uno o più campi chiave. Il termine "chiave" non va inteso nel senso in cui viene usato nella teoria relazionale, in quanto un valore della chiave non necessariamente identifica univocamente un record nel file.

Le operazioni elementari su questi tipi di file sono: *ricerca, inserimento, cancellazione e modifica*.

6.3.1 File heap

In questo tipo di organizzazione i record sono memorizzati nei blocchi senza alcun ordine: un record viene sempre inserito come ultimo record del file e l'accesso al file avviene attraverso una directory che contiene i puntatori ai blocchi (vedi Figura 6.1).

	matr	cognome	...
blocco 1	010	Rossi	...
	005	Verdi	...
	...	...	...
	031	Bianchi	...
blocco 2	...	...	...
	...	...	...
	...	...	...
	...	...	...
...	...	...	...
blocco n	...	...	...

Figura 6.1: File heap

Denotiamo con b il numero di blocchi del file ed esprimiamo in termini di b il costo delle varie operazioni:

- Se la chiave di ricerca non identifica univocamente un record nel file, poiché non esiste alcun particolare ordinamento dei record, per effettuare una ricerca occorre scandire tutto il file sequenzialmente e, quindi, il costo di una ricerca è dato da b .

Se la chiave di ricerca identifica univocamente un record nel file, la ricerca di un record con un dato valore per la chiave richiede in media $b/2$ accessi. Infatti, per la ricerca di un record che si trova nell' i -esimo blocco sono necessari i accessi (in lettura); pertanto se denotiamo con N il numero di record nel file e con R il numero di record che possono essere memorizzati in un blocco ($N/R = b$), il numero medio di accessi necessari per la ricerca di un record è data dalla somma per accedere ai singoli record diviso il numero di record:

$$\frac{R * 1 + R * 2 + \dots + R * i + \dots + R * b}{N} = \frac{R(1 + 2 + \dots + i + \dots + b)}{N} = \frac{R}{N} \frac{b(b+1)}{2} = \frac{1}{b} \frac{b(b+1)}{2} \approx \frac{b}{2}$$

- L'inserimento di un record richiede la lettura dell'ultimo blocco del file; se in tale blocco c'è spazio sufficiente per memorizzare il nuovo record, il record viene inserito, altrimenti viene chiesto un nuovo blocco al file system. Dopo l'inserimento del record nel blocco, il blocco deve essere scritto su memoria secondaria. Quindi 2 accessi (uno per lettura e uno per scrittura) sono sufficienti per l'inserimento di un record. Se non ammettiamo duplicati l'inserimento deve essere preceduto dalla ricerca, quindi dobbiamo aggiungere una media di $b/2$ accessi per verificare che non esista già un record con la chiave data.
- La cancellazione di un record con un dato valore per la chiave richiede $b/2$ accessi in lettura per la ricerca e 1 accessi in scrittura.
- La modifica di un record con un dato valore per la chiave richiede $b/2$ accessi in lettura per la ricerca e 1 accessi in scrittura.

Esempio 6.3.1 – File heap

Dati $N = 151$ record di 30 byte e dei blocchi di 65 byte che contengono dei puntatori al prossimo blocco (4 byte), calcolare il numero di blocchi necessari per memorizzare i record.

Calcoliamo il numero di record che entrano in ogni blocco:

$$\frac{(65 - 4)}{30} = \frac{61}{30} = 2.03 = 2 \text{ record per blocco}$$

Troviamo ora il numero di blocchi che occorrono per memorizzare tutti i record:

$$\frac{151}{2} = 75.5 = 76 \text{ blocchi}$$

6.3.2 File hash

In questo tipo di organizzazione i record sono ripartiti in bucket (secchi) in base al valore della chiave. Se B è il numero dei bucket, questi vengono numerati da 0 a $B - 1$. Dato un valore v per la chiave il numero del bucket in cui deve trovarsi un record con chiave v è calcolato mediante una funzione che prende il nome di *funzione hash*. Una funzione hash per essere "buona" deve ripartire uniformemente i record nei bucket, cioè al variare del valore della chiave deve assumere con la stessa probabilità uno dei valori compresi tra 0 e $B - 1$.

Ciascun bucket è costituito da uno o più blocchi ed è organizzato come un heap. L'accesso ai bucket avviene attraverso la bucket directory che contiene B elementi dove l' i -esimo elemento contiene l'indirizzo del primo blocco dell' i -esimo bucket. Tutti i blocchi di un bucket sono collegati tra loro mediante puntatori in una lista (vedi Figura 6.2).

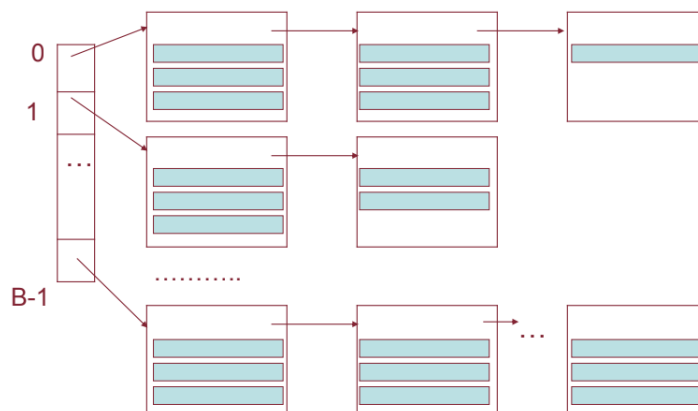


Figura 6.2: File hash

Una qualsiasi operazione (ricerca, inserimento, cancellazione, modifica) su un file hash richiede la valutazione di $h(v)$ (dove v è un valore per la chiave e h è la funzione hash) per individuare il bucket in cui deve trovarsi il record con chiave v ; successivamente, l'operazione viene effettuata sul

bucket che, come già detto, è organizzato come un heap. Se la funzione hash distribuisce uniformemente i record nei bucket, ogni bucket è costituito da un numero di blocchi che è $1/B$ – *esimo* del numero di blocchi di cui è costituito l'intero file. Pertanto il costo richiesto per un'operazione è approssimativamente $1/B$ – *esimo* del costo che sarebbe richiesto per fare la stessa operazione se il file fosse organizzato come un heap. Si osservi che, poiché l'inserimento di un record viene effettuato sull'ultimo blocco del bucket, è opportuno che la bucket directory contenga anche, per ogni bucket, un puntatore all'ultimo record del bucket.

Da quanto detto appare evidente che quanti più sono i bucket tanto più basso è il costo di ogni operazione. D'altra parte limitazioni al numero dei bucket derivano dalle seguenti considerazioni: ogni bucket deve avere almeno un blocco; se le dimensioni della bucket directory sono tali che non può essere mantenuta in memoria principale durante l'utilizzo del file, ulteriori accessi sono necessari per leggere i blocchi della bucket directory.

Esempio 6.3.2 – File hash

Supponiamo di avere un file di 250.000 record, dove ogni record occupa 300 byte, di cui 75 per il campo chiave. Ogni blocco contiene 1024 byte, di cui un puntatore al blocco che occupa 4 byte. Rispondere alle seguenti domande: (a) se usiamo una organizzazione hash con 1200 bucket, quanti blocchi occorrono per la bucket directory? (b) quanti blocchi occorrono per i bucket, assumendo una distribuzione uniforme dei record nei bucket? (c) assumendo ancora che tutti i bucket contengano il numero medio di record, qual è il numero medio di accessi a blocco per ricercare un record che sia presente nel file? (d) Quanti bucket dovremmo creare per avere invece un numero medio di accessi a blocco inferiore o al massimo uguale a 10, assumendo comunque una distribuzione uniforme dei record nei bucket?

Indichiamo con $NR = 250000$ il numero di record, $R = 300$ i byte di un record, $K = 75$ i byte del campo chiave, $CB = 1024$ i byte di un blocco, $P = 4$ i byte del puntatore.

- (a) Indichiamo con B il numero di bucket e con BD il numero di blocchi per la bucket directory. La bucket directory è essenzialmente un array di puntatori indicizzato da 0 a $B - 1$.

Vediamo prima quanti puntatori entrano in un blocco:

$$RB = \frac{CB}{P} = \frac{1024}{4} = 256$$

Ci occorreranno:

$$BD = \frac{B}{RB} = \frac{1200}{256} = 4.69 = 5$$

- (b) In un blocco dobbiamo quindi memorizzare il maggior numero possibile di record e in più un puntatore per un eventuale prossimo blocco nel bucket. Se indichiamo con M il massimo numero di record memorizzabili in un blocco, avremo:

$$M * R + P \leq CB \implies 300M + 4 \leq 1024 \quad M \leq \frac{1020}{300} = 3.4 = 3$$

Se la distribuzione dei record nei bucket è uniforme, indicando con RB il numero di record in un bucket, avremo:

$$RB = \frac{NR}{B} = \frac{250000}{1200} = 208.3 = 209$$

Indicando con NB il numero di blocchi per ogni bucket, occorrono quindi:

$$NB = \frac{RB}{M} = \frac{209}{3} = 70$$

Indicando con BB il numero complessivo di blocchi per il file hash, avremo:

$$BB = NB * B = 70 * 1200 = 84000$$

- (c) In un bucket avremo come detto $NB = RB/M = 70$ blocchi per bucket. Poiché la ricerca avviene solo sul bucket individuato in base al risultato dell'applicazione della funzione

hash alla chiave del record, avremo un numero di accessi a blocco pari a quello che si avrebbe su un heap della stessa dimensione del bucket (cioè $1/B$ rispetto alla dimensione originale). In media accederemo alla metà di questi blocchi, quindi indicando con MA il numero medio di accessi, avremo:

$$MA = \frac{NB}{2} = \frac{70}{2} = 35$$

- (d) Per avere un numero di accessi a blocco inferiore o al massimo uguale a 10, riscriviamo l'espressione di MA in modo che vi compaia esplicitamente il numero di bucket B :

$$MA = \frac{NB}{2} = \frac{\frac{RB}{M}}{2} = \frac{\frac{NR}{B}}{2} = \frac{NR}{2(B * M)} \leq 10 \quad B \geq \frac{NR}{20M} = \frac{250000}{20 * 3} = 4167$$

Verifichiamo infatti che in questo caso avremo:

$$RB = \frac{NB}{B} = \frac{250000}{4167} = 60 \quad NB = \frac{RB}{M} = 20 \quad MA = \frac{NB}{2} = 10$$

Esempio 6.3.3 – File hash

Supponiamo di avere un file di 780.000 record, dove ogni record occupa 250. Ogni blocco contiene 1024 byte e un puntatore a blocco occupa 4 byte. Usiamo una organizzazione hash con 2500 bucket. (a) Quanti blocchi dobbiamo utilizzare complessivamente per la bucket directory e per i bucket, assumendo una distribuzione uniforme dei record nei bucket? (b) Quanti blocchi dobbiamo utilizzare complessivamente per i bucket, assumendo che il 30% dei record sia distribuito in modo uniforme su 1000 bucket, e che il restante 70% dei record sia distribuito in modo uniforme sui 1500 bucket rimanenti?

Definiamo con $NR = 780000$ il numero di record, $R = 250$ i byte del record, $P = 4$ i byte del puntatore, $CB = 1024$ i byte del blocco, $B = 2500$ il numero di bucket.

- (a) Calcoliamo innanzitutto quanti puntatori entrano in ogni blocco della bucket directory:

$$PC = \frac{CB}{P} = \frac{1024}{4} = 256$$

Calcoliamo quindi quanti blocchi occorrono per la bucket directory, cioè per memorizzare 2500 puntatori:

$$BD = \frac{B}{PC} = \frac{2500}{256} = 9,7 = 10$$

Assumendo una distribuzione uniforme, dobbiamo prima di tutto calcolare quanti record devono essere memorizzati in ogni bucket:

$$RB = \frac{NR}{B} = \frac{780000}{2500} = 312$$

Vediamo ora quanti record possono essere memorizzati in un blocco, tenendo conto del fatto che ogni blocco di un bucket deve contenere anche un puntatore al blocco successivo:

$$RBL = \frac{CB - P}{R} = \frac{1020}{250} = 4.08 = 4$$

Calcoliamo infine quanti blocchi occorrono per ogni bucket:

$$BB = \frac{RB}{RBL} = \frac{312}{4} = 78$$

Complessivamente ci occorrono:

$$BD + BB * B = 10 + 78 * 2500 = 195010$$

- (b) Il 30% dei record è distribuito uniformemente su 1000 bucket, il rimanente 70% dei record è distribuito uniformemente su 1500 bucket. Quindi il numero di record che va distribuito uniformemente su 1000 bucket è:

$$N_{1000} = \frac{780000 * 30}{100} = 234000$$

Per ogni bucket dei 1000 avremo quindi:

$$RB_{1000} = \frac{N_{1000}}{1000} = 234$$

Quindi per ognuno di questi 1000 bucket occorrono:

$$B_{1000} = \frac{RB_{1000}}{RBL} = \frac{234}{4} = 58.5 = 59$$

Il numero di record che va distribuito uniformemente su 1500 bucket è:

$$N_{1500} = N - N_{1000} = 780000 - 234000 = 546000$$

Per ogni bucket dei 1500 avremo quindi:

$$RB_{1500} = \frac{N_{1500}}{1500} = 364$$

Quindi per ognuno di questi 1500 bucket occorrono quindi:

$$B_{1500} = \frac{RB_{1500}}{RBL} = \frac{364}{4} = 91$$

In totale avremo:

$$B_{1000} * 1000 + B_{1500} * 1500 = 59 * 1000 + 91 * 1500 = 195.500$$

6.3.3 File con indice (ISAM)

Nella presentazione di questo tipo di organizzazione, spesso chiamato ISAM (*Indexed Sequential Access Method*), assumeremo che il valore della chiave identifica univocamente un record del file e, inizialmente, assumeremo che i record non siano puntati.

Inizialmente il file principale viene ordinato in base al valore crescente della chiave e viene memorizzato lasciando in ogni blocco una certa percentuale (ad esempio il 20%) di spazio non utilizzato per l'inserimento di nuovi record. Quindi viene creato un nuovo file indice che contiene un record per ogni blocco del file principale e formato da due campi che contengono: un puntatore ad un blocco dal file principale e il più piccolo valore della chiave presente nel blocco (vedi Figura 6.3). L'unica eccezione è rappresentata dal primo blocco per il quale viene preso anziché il valore della chiave del primo record, un valore (denotato con $-\infty$) che è più piccolo di qualsiasi valore che può essere assunto dalla chiave.

Identifichiamo ora il costo delle varie operazioni su questa struttura:

- I record del file indice possono a loro volta essere ordinati in base al valore della chiave, in questo modo è possibile utilizzare la *ricerca binaria*. Siano $B_1, B_2, \dots, B_m$ i blocchi del file indice; si comincia con il considerare il blocco $B_{\lfloor m/2 \rfloor}$ e si confronta il valore w della chiave nel primo record di tale blocco con v ; se $v = w$ la ricerca ha termine; se $v < w$ si ripete il procedimento come se il file indice fosse costituito dai blocchi $B_1, B_2, \dots, B_{\lfloor m/2-1 \rfloor}$; se invece $v > w$ si ripete il procedimento come se il file indice fosse costituito dai blocchi $B_{\lfloor m/2 \rfloor}, B_{\lfloor m/2+1 \rfloor}, \dots, B_m$. Il procedimento viene iterato fino a quando ci si riduce a considerare un unico blocco; a questo punto si scandisce il blocco per trovare un valore della chiave che ricopre v . Poiché ad ogni passo il numero di blocchi su cui deve essere effettuata la ricerca viene dimezzato, dopo al più $\log_2(m)$ passi la ricerca è ristretta ad un unico blocco.

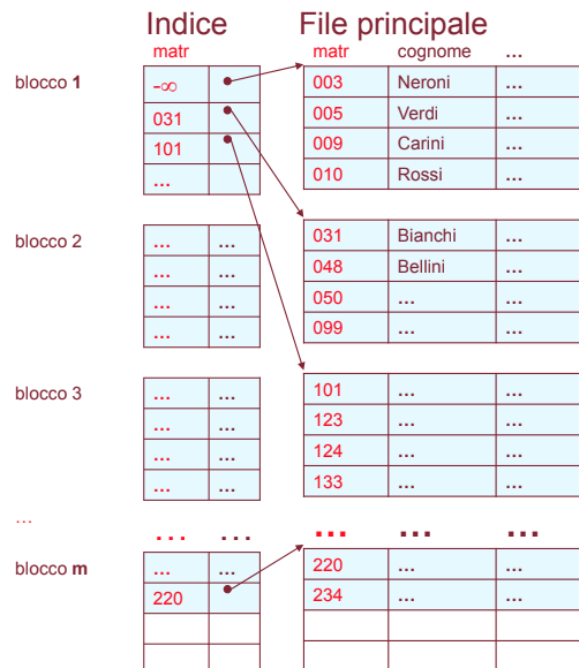


Figura 6.3: File con indice

Un altro metodo è costituito dalla *ricerca per interpolazione*. Questo tipo di ricerca è basato sulla conoscenza della distribuzione dei valori della chiave. In altre parole per poter effettuare questa ricerca si deve disporre di una funzione f che dati tre valori v_1, v_2, v_3 della chiave fornisce un valore che è la frazione dell'intervallo di valori della chiave compresi tra v_2 e v_3 in cui deve trovarsi v_1 . In tal caso, se $B_1, B_2, \dots, B_m$ sono i blocchi del file indice (o di una sua parte) e v_2 è il valore della chiave nel primo record di B_1 e v_3 è il valore della chiave nell'ultimo record di B_m allora v_1 deve essere confrontato con il valore della chiave nel primo record di B_i , dove $i = f(v_1, v_2, v_3) * m$; analogamente a quanto accade nella ricerca binaria, se v_1 è minore di tale valore allora il procedimento deve essere ripetuto sui blocchi $B_1, B_2, \dots, B_{i-1}$, mentre se è maggiore il procedimento deve essere ripetuto sui blocchi $B_i, B_{i+1}, \dots, B_m$, finché la ricerca si restringe ad un unico blocco. E' stato mostrato che la ricerca per interpolazione richiede circa $1 + \log_2 \log_2(m)$ accessi.

- Per inserire un nuovo record nel file principale occorre innanzitutto ricercare il blocco B_i del file principale in cui il record deve trovarsi in base al valore della chiave. Se in B_i c'è spazio sufficiente, il record viene inserito in modo da rispettare l'ordinamento (quindi spostando eventualmente record con chiave maggiore). Altrimenti si possono seguire diverse strategie. Ad esempio si può verificare se c'è spazio sufficiente nel blocco successivo B_{i+1} ; in tal caso, o il nuovo record o un record che era precedentemente in B_i diventa il primo record di B_{i+1} e, pertanto, il record del file indice che contiene il puntatore a B_{i+1} deve essere modificato. Se B_{i+1} non esiste (B_i è l'ultimo blocco del file) oppure è pieno si può prendere in considerazione il blocco B_{i-1} ed eventualmente procedere in modo simile ripartendo i record di tra B_{i-1} e B_i ; anche in questo caso sarà necessario apportare una modifica al file indice. Se anche B_{i-1} è pieno oppure se B_{i-1} non esiste (B_i è il primo blocco del file), è necessario richiedere al sistema un nuovo blocco che seguirà B_i e ripartire i record di B_i tra B_i e il nuovo blocco; quindi occorre inserire nel file indice il record relativo al nuovo blocco.
- Per cancellare un record dal file principale occorre innanzitutto ricercare il record, quindi cancellarlo, spostare i record nel blocco in modo da recuperare lo spazio lasciato libero. Se il record cancellato è il primo record del blocco occorre modificare nel file indice il record relativo al blocco. Se il record cancellato era l'unico record del blocco, il blocco viene restituito al sistema e nel file indice occorre cancellare il record relativo al blocco.
- Per modificare un record del file principale occorre innanzitutto ricercare il record; quindi, se la modifica non coinvolge campi della chiave, il record viene modificato e il blocco riscritto.

Altrimenti la modifica equivale ad una cancellazione seguita da un inserimento.

Consideriamo ora il caso in cui il file principale contiene record puntati.

In tal caso la fase di inizializzazione non differisce dal caso precedente salvo che è preferibile lasciare più spazio libero nei blocchi per successivi inserimenti (vedi Figura 6.4). Infatti, poiché i record sono puntati, non possono essere spostati per mantenere l'ordinamento. Le operazioni saranno:

- La ricerca di un record con chiave v richiede la ricerca sul file indice di un valore della chiave che ricopre v e quindi la scansione del bucket corrispondente.
- La cancellazione di un record richiede la ricerca del record e quindi la modifica dei bit di cancellazione nell'intestazione del blocco.
- La modifica di un record richiede la ricerca del record; quindi, se la modifica non coinvolge campi della chiave, il record viene modificato e il blocco riscritto. Altrimenti la modifica equivale ad una cancellazione seguita da un inserimento. In quest'ultimo caso non è sufficiente modificare il bit di cancellazione del record cancellato, ma è necessario inserire in esso un puntatore al nuovo record inserito in modo che questo sia raggiungibile da qualsiasi record che contenga un puntatore al record cancellato.

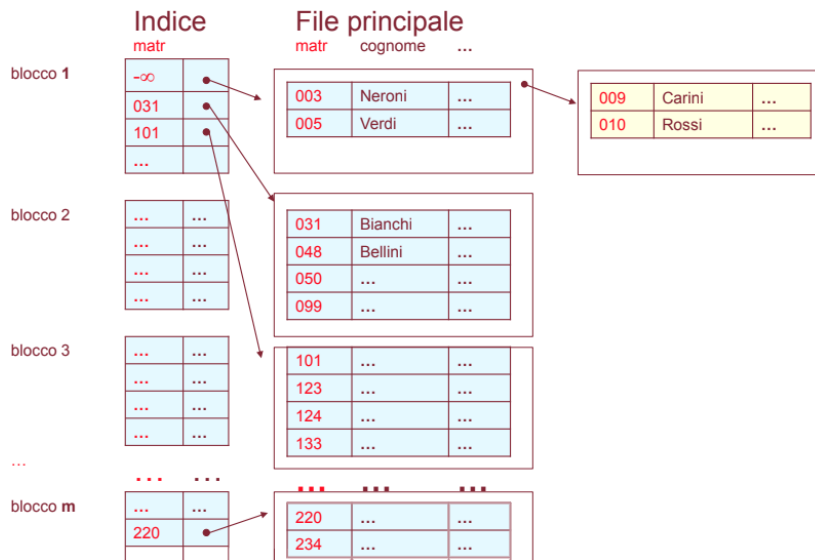


Figura 6.4: File con indice con record puntati

Poiché non è possibile mantenere il file principale ordinato, se si vuole avere la possibilità di esaminare il file seguendo l'ordinamento della chiave occorre inserire in ogni record un puntatore al record successivo nell'ordinamento.

Esempio 6.3.4 – File con indice

Supponiamo di avere un file di 150.000 record. Ogni record occupa 250 byte, di cui 50 per il campo chiave. Ogni blocco contiene 1024 byte. Un puntatore a blocco occupa 4 byte.

(a) Se usiamo un indice ISAM sparso, e assumiamo che i record non siano puntati e che il fattore di utilizzo dei blocchi del file principale sia 0,7 (cioè i blocchi non sono completamente pieni, ma pieni al più al 70%), quanti blocchi dobbiamo utilizzare per l'indice? (b) Se usiamo un indice ISAM sparso, e assumiamo che i record siano puntati e che i blocchi del file principale siano pieni, quanti blocchi dobbiamo utilizzare per l'indice? (c) Se utilizziamo la ricerca binaria, quale è il numero massimo di accessi a blocco per ricercare un record presente nel file nei casi (a) e (b), supponendo nel caso (b) di non avere liste di overflow?

- (a) I record sono di taglia fissa, quindi non occorrono puntatori all'inizio del blocco; poiché i record non sono puntati, nei blocchi del file principale non occorre un puntatore al prossimo blocco. L'esercizio fornisce il fattore di occupazione attuale dei blocchi del file del file principale, ma non indica un fattore di occupazione per i blocchi indice, quindi

possiamo assumere che siano pieni.

Dobbiamo stabilire quanti record indice occorrono, quindi sapendo che l'indice contiene un record per ogni blocco del file principale, dobbiamo calcolare quanti sono questi blocchi. Sappiamo che i blocchi dati sono occupati al più al 70% (indichiamo questa quantità con PO), quindi prima di tutto vediamo quanti record interi possono essere contenuti al massimo nella porzione indicata di blocco. Indicando con M questo numero, avremo che:

$$M = \frac{(CB * PO)}{R} = \frac{(1024 * \frac{70}{100})}{250} = 2.86 \approx 2$$

Il numero di blocchi da indicizzare, indicato con BF, sarà quindi:

$$BF = \frac{NR}{M} = \frac{150000}{2} = 75000$$

Ogni record indice è composto da chiave e puntatore al blocco, quindi occupa $RI = K + P$ byte, cioè $RI = 54$ byte. Dobbiamo ora calcolare quanti record indice interi possono essere contenuti in un blocco pieno, indicando con MI questo numero avremo:

$$MI = \frac{CB}{RI} = \frac{1024}{54} = 18$$

In definitiva, indicando con BI il numero di blocchi indice, avremo:

$$BI = \frac{BF}{MI} = \frac{75000}{18} = 4167$$

- (b) L'esercizio ci dice in questo caso che i blocchi dati sono pieni, e lo stesso possiamo assumere per i blocchi indice. Ogni blocco del file principale deve essere visto come parte di un bucket, e quindi dobbiamo anche prevedere spazio per un puntatore. Vediamo prima di tutto in queste nuove condizioni quanti record interi di dati entrano al massimo in un blocco:

$$M = \frac{(CP - P)}{R} = \frac{1020}{250} = 4.08 \approx 4$$

Vediamo allora quanti blocchi vanno indicizzati indicando questo numero con BF avremo:

$$BF = \frac{NR}{M} = \frac{150000}{4} = 37500$$

A questo punto calcoliamo il numero di blocchi indice. La struttura dl blocco indice è identica a quella del caso precedente, quindi ricordiamo che la taglia del record indice è $RI = 54$ byte, che il numero massimo di record indice per blocco è $MI = 18$ e quindi occorrono:

$$BI = \frac{BF}{MI} = \frac{37500}{18} = 2084$$

- (c) In entrambi i casi la ricerca si effettua prima di tutto sui blocchi indice; assumiamo una ricerca binaria; i due casi si differenziano, perché nel caso di record non puntati dobbiamo ancora leggere in memoria un solo blocco di record di dati, mentre nel caso di record puntati l'indice punta ad un bucket che potrebbe contenere più blocchi di overflow, che vanno tutti esaminati. Nel nostro caso però l'esercizio dice esplicitamente che non ci sono liste di overflow, quindi in entrambe le configurazioni dobbiamo aggiungere un solo accesso a blocco, quindi nella configurazione (a) e (b) avremo rispettivamente:

$$A = \log_2 BI + 1 = \log_2 4167 + 1 = 14$$

$$A = \log_2 BI + 1 = \log_2 2084 + 1 = 13$$

6.3.4 File B-tree

Questa organizzazione è una generalizzazione del file con indice, infatti in un B-tree si accede al file attraverso una gerarchia di indici. L'indice a livello più alto nella gerarchia è costituito da un unico blocco e quindi può risiedere in memoria principale durante l'utilizzo del file.

Ogni blocco di un file indice è costituito di record contenenti una coppia (v, b) dove v è il valore della chiave del primo record della porzione del file principale che è accessibile attraverso il puntatore b ; b può essere un puntatore ad un blocco del file indice a livello immediatamente più basso oppure (nei record del file indice nel più basso livello della gerarchia) ad un blocco del file principale. I blocchi degli indici e del file principale devono essere sempre pieni almeno per metà (vedi Figura 6.5).

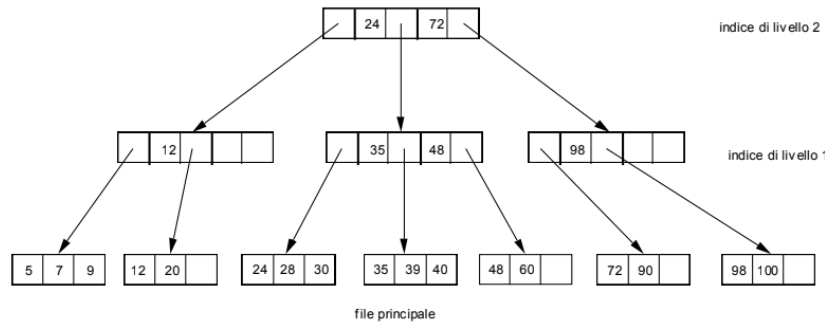


Figura 6.5: File B-tree

- Durante la ricerca di un record con un dato valore per la chiave si accede agli indici a partire da quello a livello più alto; a mano a mano che si scende nella gerarchia di indici si restringe la porzione (insieme di blocchi) del file principale in cui deve trovarsi il record desiderato, fino a che, nell'ultimo livello (il più basso nella gerarchia) tale porzione è ristretta ad un unico blocco.

Più in dettaglio, per ricercare il record del file principale con un dato valore v per la chiave si procede nel modo seguente. Si parte dall'indice a livello più alto (che è costituito da un unico blocco) e ad ogni passo si esamina un unico blocco. Se il blocco esaminato è un blocco del file principale, tale blocco è quello in cui deve trovarsi il record desiderato; se, invece, è un blocco di un file indice, si cerca su tale blocco un valore della chiave che ricopre v e si segue il puntatore associato (che sarà o un puntatore ad un blocco dell'indice al livello immediatamente inferiore o un blocco del file principale). Per valutare il costo di un'operazione di ricerca, assumiamo, per comodità, che il numero di record del file principale che possono essere memorizzati in un blocco sia un numero dispari $2e - 1$, che il numero di record di un file indice che possono essere memorizzati in un blocco sia un numero dispari $2d - 1$ (cioè, poiché ogni blocco è pieno almeno per metà, ogni blocco del file principale contiene almeno e record e ogni blocco di un indice contiene almeno d record). Con tali assunzioni il file principale ha al più n/e blocchi, dove n è il numero di record del file principale. Poiché ad ogni blocco del file principale corrisponde un record nel file indice al livello 1 (il più basso nella gerarchia) il numero di record del file indice a livello più basso sarà n/e e, per l'ipotesi fatta (ogni blocco è pieno almeno per metà), questi n/e record saranno memorizzati in al più $n/(ed)$ blocchi. Ripetendo tale ragionamento, si ottiene che l'indice a livello i ha $n/(ed^{i-1})$ record memorizzati in al più $n/(ed^i)$ blocchi. Sia k il livello più alto della gerarchia; l'indice a livello k avrà $n/(ed^{k-1})$ record memorizzati in al più $n/(ed^k)$ blocchi; poiché tale indice è costituito da un unico blocco $n/(ed^k) \geq 1$. Poiché per ricercare un record del file principale si deve accedere ad un blocco per ogni livello di indice più al blocco del file principale che contiene il record desiderato, il numero di accessi necessari per un operazione di ricerca è $k + 1 \leq \log_d(n/e) + 1$.

- Il numero di accessi necessari per effettuare l'inserimento del record con valore v della chiave è dato dal numero di accessi necessari per ricercare il blocco in cui deve essere inserito il record con chiave v più uno per riscrivere il blocco, se nel blocco c'è spazio sufficiente per inserire il record. Se, viceversa, nel blocco non c'è spazio sufficiente per inserire il nuovo record, l'operazione di inserimento può richiedere un numero ulteriore di accessi a causa della necessità di mantenere i blocchi pieni al meno per metà.

- Il numero di accessi necessari per effettuare la cancellazione del record con valore v della chiave è dato dal numero di accessi necessari per ricercare il blocco in cui si trova il record con chiave v più uno per riscrivere il blocco, se il blocco rimane pieno almeno per metà successivamente alla cancellazione. Se, viceversa, il blocco non rimane pieno almeno per metà successivamente alla cancellazione, l'operazione può richiedere un numero ulteriore di accessi.
- Il numero di accessi necessari per effettuare la modifica del record con valore v della chiave è dato dal numero di accessi necessari per ricercare il blocco in cui si trova il record con chiave v più uno per riscrivere il blocco, se la modifica non coinvolge campi della chiave, altrimenti è dato dalla somma degli accessi necessari per la cancellazione del record con chiave v e quelli necessari per l'inserimento del record con il nuovo valore della chiave.

Notiamo che se il numero di record massimo che possiamo memorizzare è dispari, quindi esprimibile nella forma $2E - 1$, possiamo direttamente considerare E come occupazione minima. Se il numero massimo di record che possiamo memorizzare è pari, quindi esprimibile come $2E$, possiamo considerare $E + 1$ come occupazione minima.

Esempio 6.3.5 – File B-tree

Supponiamo di avere un file di 170.000 record dove ogni record occupa 200 byte, di cui 20 per il campo chiave. Ogni blocco contiene 1024 byte e un puntatore a blocco occupa 4 byte. Se usiamo un B-tree e assumiamo che sia i blocchi indice che i blocchi del file sono pieni al minimo, quanti blocchi vengono usati per il livello foglia (file principale) e quanti per l'indice, considerando tutti i livelli non foglia? Qual'è il costo di una ricerca in questo caso?

Per quanto riguarda i blocchi di dati nel livello foglia (file principale) indichiamo il numero massimo di record memorizzabili con MR, avremo:

$$MR = \frac{CB}{R} = \frac{1024}{200} = 5$$

Poiché il numero in questo caso è dispari ($MR = 2E - 1 = 5$) per definizione, un blocco dati non conterrà mai meno di $E = 3$ record. A questo punto possiamo calcolare il numero di blocchi nel livello foglia, che sarà:

$$BF = \frac{NR}{E} = \frac{170000}{3} = 56667$$

A questo punto dobbiamo calcolare la capacità dei blocchi indice. Nei livelli indice del B-tree, ogni record, eccetto il primo, contiene una coppia (chiave, puntatore); il primo record invece contiene solo un puntatore (relativo al blocco dati che contiene i record con chiave minore della chiave contenuta nel record indice successivo). Indicando con $MaxI$ tale numero dovremo avere:

$$(MaxI - 1) * K + (MaxI) * P \leq CB \iff (MaxI - 1) * 20 + (MaxI) * 4 \leq 1024 \iff \\ MaxI \leq \frac{1044}{24} = 43.5 \approx 43$$

Volendo il minimo numero di blocchi abbiamo $2D - 1 = MaxI \iff 2 * D - 1 = 43 \iff D = 22$. Verifichiamo però che la metà dei record riempia la metà del blocco, considerando che il primo è solo puntatore: $21 * (20 + 4) + 4 = 508$ che è meno della metà del blocco quindi per l'occupazione minima aggiungiamo 1 record (arriviamo a occupare 532). Quindi l'occupazione minima sarà $MinI = 23$ record indice per blocco indice.

Al primo livello di indice avremo quindi $B1 = \frac{BF}{MinI} = \frac{56667}{23} = 2463.78 = 2464$ blocchi indice. Ad ognuno di questi corrisponderà un record al secondo livello di indice, quindi $B2 = \frac{B1}{MinI} = \frac{2464}{23} = 107.13 = 108$ blocchi indice. Iterando il ragionamento avremo $B3 = \frac{B2}{MinI} = \frac{108}{23} = 5$ blocchi indice. A questo punto attenzione: anche questi ultimi sono blocchi a cui devono corrispondere dei record indice al livello superiore. Arriviamo così alla radice, che deve contenere sempre un unico blocco, e per la quale si rilascia il vincolo che sia piena almeno per metà (infatti nel nostro caso conterrà solo 5 record). Abbiamo allora $B4 = 1$ blocco indice.

In conclusione abbiamo $BF = 56.667$ blocchi dati al livello foglia, e complessivamente $BI = B1 + B2 + B3 + B4 = 2464 + 108 + 5 + 1 = 2578$ blocchi nei 4 livelli di indice.

Il costo di una ricerca sarà di 5 accessi (un blocco per ognuno dei 4 livelli di indice + 1 blocco del file principale).

Capitolo 7

Controllo della concorrenza

La maggior parte della teoria del controllo della concorrenza nelle basi di dati è stata sviluppata per sistemi di calcolo con una sola CPU. In tali sistemi i programmi sono eseguiti concorrentemente in modo interleaved (interfogliato): la CPU può eseguire un solo programma alla volta, tuttavia il sistema operativo permette di eseguire alcune istruzioni di un programma, sospendere quel programma, eseguire istruzioni di un altro programma e quindi ritornare ad eseguire istruzioni del primo.

7.1 Introduzione alla concorrenza

In un sistema di gestione di basi di dati multiutente la principale risorsa a cui i vari programmi accedono concorrentemente è la base di dati. L'esecuzione di una parte di un programma che rappresenta un'unità logica di accesso o modifica del contenuto della base di dati è detta transazione. Nei sistemi in cui vengono effettuate da più utenti sia operazioni di lettura che di scrittura l'esecuzione concorrente di più transazioni può provocare problemi se non viene controllata in qualche modo. Perché le transazioni operino in modo corretto sui dati è necessario che i meccanismi che le implementano soddisfino queste quattro proprietà:

- *Atomicità*: la transazione è indivisibile nella sua esecuzione, infatti la sua esecuzione deve essere o totale o nulla e non sono ammesse esecuzioni parziali;
- *Consistenza*: quando inizia una transazione il database si trova in uno stato consistente e quando la transazione termina il database deve essere in un altro stato consistente, ovvero non deve violare eventuali vincoli di integrità;
- *Isolamento*: ogni transazione deve essere eseguita in modo isolato e indipendente dalle altre transazioni, l'eventuale fallimento di una transazione non deve interferire con le altre transazioni in esecuzione;
- *Durabilità*: si riferisce al fatto che una volta che una transazione abbia richiesto un commit work, i cambiamenti apportati non dovranno essere più persi. Per evitare che nel lasso di tempo fra il momento in cui la base di dati si impegna a scrivere le modifiche e quello in cui li scrive effettivamente si verifichino perdite di dati dovuti a malfunzionamenti, vengono tenuti dei registri di log dove sono annotate tutte le operazioni sul DB.

Prima di esaminare alcuni dei problemi che possono sorgere quando l'esecuzione concorrente di più transazioni non è controllata, introduciamo il concetto di *schedule* (piano di esecuzione) di un insieme di transazioni. Dato un insieme T di transazioni uno *schedule* S di T è un ordinamento delle operazioni nelle transazioni in T tale che per ogni transazione T in T se o_1 e o_2 sono due operazioni in T tali che o_1 precede o_2 in T allora o_1 precede o_2 in S (in altre parole uno *schedule* deve conservare l'ordine che le operazioni hanno all'interno delle singole transazioni). Qualsiasi *schedule* ottenuto permutando le transazioni in T è detto *seriale*.

Consideriamo le seguenti transazioni e i seguenti schemi di T_1, T_2 :

T_1	T_2	T_1	T_2	T_1	T_3
read(X) $X:=X-N$		read(X) $X:=X-N$ write(X)		read(X) $X:=X-N$ write(X)	somma:=0
	read(X) $X:=X+M$		read(X) $X:=X+M$ write(X)		read(X) somma:=somma+X
write(X) read(Y)					read(Y) somma:=somma+Y
$Y:=Y+N$ write(Y)	write(X)	read(Y) T_1 fallisce	write(X)	read(Y) $Y:=Y+N$ write(Y)	

Nel primo caso l'aggiornamento di X prodotto da T_1 viene perso in quanto T_2 legge il valore di X prima che l'aggiornamento prodotto da T_1 sia stato reso permanente. Nel secondo caso T_2 legge e aggiorna il valore di X dopo che l'aggiornamento prodotto da T_1 è stato reso permanente, ma prima che venga ripristinato il vecchio valore di X in conseguenza del fallimento di T_1 . Consideriamo ora la transazione T_3 che somma i valori di X e di Y. Il seguente schedule di fa sì che la somma prodotta da T_3 sia la somma del valore di X dopo che X è stato aggiornato da T_1 e del valore di Y prima che sia stato aggiornato da T_1 .

In tutti e tre i casi visti siamo portati a considerare gli schedule non corretti in quanto i valori prodotti non sono quelli che si avrebbero se le due transazioni fossero eseguite nel modo "naturale" cioè sequenzialmente. In generale possiamo osservare che l'esecuzione naturale e, quindi, intuitivamente corretta di un insieme di transazioni è quella sequenziale; la possibilità di eseguire concorrentemente un insieme di transazioni, come si è detto, è introdotta nei sistemi per motivi di efficienza. Pertanto tutti gli schedule seriali sono corretti e uno schedule non seriale è corretto se è serializzabile, cioè se è "equivalente" ad uno schedule seriale. Sorge quindi la necessità di definire un concetto di equivalenza di schedule.

Due schedule sono *equivalenti* se (per ogni dato modificato) producono valori uguali, dove due valori sono uguali solo se sono prodotti dalla stessa sequenza di operazioni. Ad esempio i seguenti due schedule non sono considerati equivalenti:

T_1	T_2	T_1	T_2
read(X) $X:=X+N$ write(X)			read(X) $X:=X-M$ write(X)
	read(X) $X:=X-M$ write(X)	read(X) $X:=X+N$ write(X)	

Oltre alla definizione di equivalenza, un altro elemento che ha influenza sulla complessità del problema di decidere se uno schedule è *serializzabile* (cioè se è equivalente ad uno schedule seriale). Nella pratica è difficile testare la serializzabilità di uno schedule. Infatti l'ordine di esecuzione delle operazioni delle diverse transazioni è determinato in base a diversi fattori: il carico del sistema, l'ordine temporale in cui le transazioni vengono sottomesse al sistema e le loro priorità. Pertanto è praticamente impossibile determinare in anticipo come le operazioni saranno interleaved, cioè in quale ordine verranno eseguite; d'altra parte, se prima si eseguono le operazioni e poi si testa la serializzabilità dello schedule, i suoi effetti devono essere annullati se lo schedule risulta non serializzabile. Quindi l'approccio seguito nei sistemi è quello di determinare metodi che garantiscano la serializzabilità di uno schedule eliminando così la necessità di dover testare ogni volta la serializzabilità di uno schedule. Uno di tali metodi consiste nell'imporre dei protocolli, cioè delle regole, alle transazioni in modo da garantire la serializzabilità di ogni schedule. Questi protocolli usano tecniche di locking (cioè di controllo dell'accesso ai dati) per prevenire l'accesso concorrente ai dati. Altri metodi di controllo usano i timestamp delle transazioni, cioè degli identificatori delle transazioni che vengono generati dal sistema e in base ai quali le operazioni delle transazioni possono essere ordinate in modo da assicurare la serializzabilità.

7.2 Item

Tutte le tecniche per la gestione della concorrenza richiedono che la base di dati sia partizionata in item, cioè in unità a cui l'accesso è controllato. Le dimensioni degli item devono essere definite in base all'uso che viene fatto della base di dati in modo tale che in media una transazione acceda a pochi item. Le dimensioni degli item usate da un sistema sono dette la sua granularità. Una granularità grande permette una gestione efficiente della concorrenza; una piccola granularità può invece sovraccaricare il sistema, ma consente l'esecuzione concorrente di molte transazioni.

7.3 Locking

Un lock viene richiesto da una transazione mediante un'operazione di locking e viene rilasciato mediante un'operazione di unlocking; fra l'esecuzione di un'operazione di locking su un certo item X e l'esecuzione di un'operazione di unlocking su X diciamo che la transazione mantiene un lock su X. Il locking agisce come primitiva di sincronizzazione, cioè se una transazione richiede un lock su un item su cui un'altra transazione mantiene un lock, la transazione non può procedere finché il lock non viene rilasciato dalla prima transazione. Uno schedule è detto legale se una transazione effettua un locking ogni volta che deve leggere o scrivere un item e ciascuna transazione rilascia ogni lock che ha ottenuto.

7.3.1 Lock binario

Un lock binario può assumere solo due valori *locked* e *unlocked*. Le transazioni fanno uso di due operazioni `lock(X)` e `unlock(X)`; la prima serve per richiedere l'accesso all'item X, la seconda per rilasciare l'item X consentendone l'accesso ad altre transazioni.

Consideriamo di nuovo le due transazioni dell'esempio precedente e vediamo come l'uso dei lock può prevenire il problema dell'"aggiornamento perso":

T_1	T_2
<code>lock(X)</code> <code>read(X)</code> <code>X:=X-N</code> <code>write(X)</code> <code>unlock(X)</code>	<code>lock(X)</code> <code>read(X)</code> <code>X:=X+M</code> <code>write(X)</code> <code>unlock(X)</code>
<code>lock(Y)</code> <code>read(Y)</code> <code>Y:=Y+N</code> <code>write(Y)</code> <code>unlock(Y)</code>	

Il più semplice modello per le transazioni è quello che considera una transazione come una sequenza di operazioni di lock e unlock. Si assume che ogni operazione di lock su un item X implica la lettura di X e ogni operazione di unlock di un item X implica la scrittura di X. Il nuovo valore dell'item viene calcolato da una funzione che è associata in modo univoco ad ogni coppia lock-unlock ed ha per argomenti tutti gli item letti (locked) dalla transazione prima dell'operazione di unlock.

Due schedule sono equivalenti se le formule che danno i valori finali per ciascun item sono le stesse per i due schedule. Consideriamo ora le due transazioni e i seguenti schedule di T_1, T_2 :

T_1	T_2
lock(X)	lock(Y)
unlock(X) $f_1(X)$	unlock(Y) $f_3(Y)$
lock(Y)	lock(X)
unlock(Y)	unlock(X)
$f_2(X,Y)$	$f_4(X,Y)$

T_1	T_2
lock(X)	lock(Y) unlock(Y)
unlock(X)	
lock(Y)	lock(X) unlock(X)
unlock(Y)	

Se indichiamo con X_0 e Y_0 i valori iniziali di X e Y, abbiamo che il valore finale per X prodotto da S è dato dalla formula $f_4(f_1(X_0), Y_0)$. D'altra parte il valore finale per X prodotto dallo schedule seriale T_1, T_2 è dato dalla formula $f_4(f_1(X_0), f_2(X_0, Y_0))$, mentre il valore finale per X prodotto dallo schedule seriale T_2, T_1 è dato dalla formula $f_1(f_4(X_0, Y_0))$; pertanto S non è serializzabile in quanto non è equivalente a nessuno schedule seriale di T_1, T_2 .

Consideriamo ora le due transazioni e i seguenti schedule di T_1, T_2 :

T_1	T_2
lock(X)	lock(X)
unlock(X) $f_1(X)$	unlock(X) $f_3(X)$
lock(Y)	lock(Y)
unlock(X)	unlock(Y)
$f_2(X,Y)$	$f_4(X,Y)$

T_1	T_2
lock(X)	lock(X) unlock(X)
unlock(X)	
lock(Y)	lock(Y) unlock(Y)
unlock(Y)	

Se indichiamo con X_0 e Y_0 i valori iniziali di X e Y, abbiamo che il valore finale per X e Y prodotti da S sono dati rispettivamente dalle formule $f_3(f_1(X_0))$ e $f_4(f_1(X_0), f_2(X_0, Y_0))$. Poiché tali formule coincidono con quelle prodotte dallo schedule seriale T_1, T_2 lo schedule S è serializzabile.

La serializzabilità di uno schedule in questo semplice modello, può essere testata mediante un semplice algoritmo su grafi diretti. L'idea su cui si basa tale algoritmo è la seguente. Dato uno schedule, per ogni item si esamina l'ordine in cui le varie transazioni fanno un lock su quell'item; se lo schedule è serializzabile questo ordine deve essere consistente con quello di uno schedule seriale; pertanto se l'ordine imposto sulle transazioni da un certo item è diverso da quello imposto da un altro item, lo schedule non è serializzabile.

Algoritmo 7.3.1 – Serializzabilità di uno schedule

Dato uno schedule S:

- Crea un grafo diretto G (grafo di serializzazione) i cui nodi corrispondono alle transazioni; in tale grafo c'è un arco diretto da T_i a T_j (con etichetta X) se in S T_i esegue un unlock(X) e T_j esegue il successivo lock(X);
- Se G ha un ciclo allora S non è serializzabile; altrimenti applicando a G l'ordinamento topologico (eliminando ricorsivamente i nodi del grafo che non hanno archi entranti) si ottiene uno schedule seriale S' equivalente ad S.

Teorema 7.3.1 – Correttezza dell'Algoritmo del grafo di serializzazione

Uno schedule S è serializzabile se e solo se il suo grafo di serializzazione è aciclico.

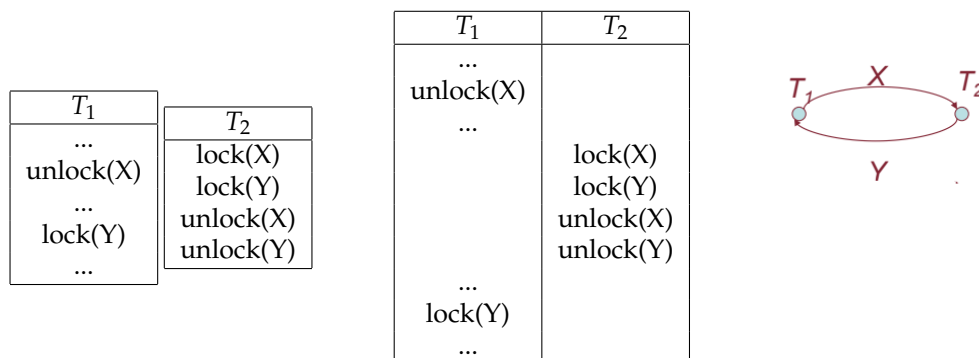
Diciamo che una transazione obbedisce al protocollo di *locking a due fasi*, o più semplicemente che è a due fasi, se prima effettua tutte le operazioni di lock (fase di locking) e poi tutte le operazioni di unlock (fase di unlocking).

Teorema 7.3.2 – Locking a due fasi è serializzabile

Sia T un insieme di transazioni. Se ogni transazione in T è a due fasi allora ogni schedule di T è serializzabile.

Dimostrazione. Sia S uno schedule di T . Supponiamo, per assurdo, che S non sia serializzabile. Per il Teorema 7.3.1, il grafo di serializzazione G di S contiene un ciclo $T_1, T_2, \dots, T_k, T_1$; ciò significa che T_{i+1} , $i = 1, \dots, k-1$, effettua un'operazione di lock, su un item su cui T_i ha effettuato un'operazione di unlock e che T_1 effettua un'operazione di lock, su un item su cui T_k ha effettuato un'operazione di unlock. Pertanto in S T_1 effettua un'operazione di lock dopo aver effettuato un'operazione di unlock e quindi T_1 non è a due fasi (contraddizione).

Mostreremo ora che se una transazione T_1 non è a due fasi, esiste sempre una transazione T_2 tale che esiste uno schedule di T_1, T_2 che non è serializzabile. Infatti, se T_1 non è a due fasi effettua un'operazione di lock dopo aver effettuato un'operazione di unlock:



Naturalmente possono esistere schedule di transazioni che non sono a due fasi che sono serializzabili. Tuttavia poiché è normale non conoscere l'insieme di transazioni con cui una transazione può essere eseguita concorrentemente siamo costretti a richiedere che tutte le transazioni siano a due fasi perché sia garantita la serializzabilità di ogni schedule.

7.4 Deadlock e livelock

In un sistema che utilizza i lock per il controllo della concorrenza si possono verificare delle situazioni anomale note con il nome di deadlock e livelock.

Un livelock si verifica quando una transazione aspetta indefinitamente che gli venga garantito un lock su un certo item.

Un deadlock si verifica quando ogni transazione in un insieme T è in attesa di ottenere un lock su un item sul quale qualche altra transazione in T mantiene un lock.

Il primo problema può essere risolto con una strategia first come-first served; in altre parole il sistema mantiene memoria delle successive richieste di lock in modo che, quando un item X viene rilasciato, viene garantito un lock su X alla transazione che per prima ha richiesto un lock su X . Un'altra possibile strategia può essere quella di eseguire le transazioni in base alle loro priorità e di aumentare la priorità di una transazione all'aumentare del tempo in cui rimane in attesa in modo che possa ad un certo punto assumere la priorità più alta ed essere eseguita.

Per risolvere il secondo problema possono essere seguiti due approcci. Un approccio è preventivo in quanto cerca di evitare il verificarsi di situazioni di stallo adottando opportuni protocolli; l'altro invece si preoccupa di risolvere le situazioni di stallo quando si verificano. Il sussistere di una situazione di stallo può essere rilevato mantenendo un grafo di attesa, cioè un grafo i cui nodi sono le transazioni e in cui c'è un arco $T_1 \rightarrow T_2$ se la transazione T_1 è in attesa di ottenere un lock su un item sul quale T_2 mantiene un lock; se in tale grafo c'è un ciclo si sta verificando una situazione di stallo che coinvolge le transazioni nel ciclo; per risolverla occorre che almeno una transazione nel ciclo sia rolled-back (cioè la transazione viene abortita, i suoi effetti sulla base di dati vengono annullati ripristinando i valori dei dati precedenti l'inizio della sua esecuzione e, infine, tutti i lock mantenuti dalla transazione vengono rilasciati) e quindi venga fatta ripartire.

7.5 Locking a due fasi stretto

Oltre al verificarsi di un deadlock, ci sono altri motivi per cui una transazione deve essere abortita, ad esempio: perché ha cercato di effettuare un accesso non autorizzato oppure perché ha cercato di eseguire una divisione per 0. Il punto in cui una transazione non può più essere abortita per uno dei suddetti motivi, cioè il punto in cui ha ottenuto tutti i lock che gli sono necessari e ha effettuato tutti i calcoli nell'area di lavoro, viene detto punto di commit della transazione; una transazione che ha raggiunto il suo punto di commit viene detta committed altrimenti viene detta attiva. Quando una transazione raggiunge il suo punto di commit effettua un'operazione di commit (nei sistemi reali tale operazione prevede lo svolgimento di diverse azioni, ma per i nostri scopi serve solo a marcare il punto di commit della transazione). I dati scritti da una transazione sulla base di dati prima che abbia raggiunto il punto di commit vengono detti dati sporchi. Il fatto che un dato sporco possa essere letto da qualche altra transazione può causare un effetto di roll-back a cascata. Consideriamo il seguente schedule:

T_1	T_2
wlock(X)	
read(X)	
$X := X - N$	
write(X)	
unlock(X)	
	rlock(X)
	wlock(Z)
	read(X)
	read(Z)
	$Z := Z + X$
	write(Z)
	commit
	unlock(X)
	unlock(Z)
wlock(Y)	
read(Y)	
$Y := Y + N$	
write(Y)	
commit	
unlock(Y)	

Se la transazione T_1 viene abortita dopo che ha letto Y , il valore di X scritto da T_1 è un dato sporco; infatti, quando T_1 viene abortita è necessario annullare gli effetti di T_1 sulla base di dati ripristinando il vecchio valore di X (quello letto da T_1). Ma allora è necessario annullare anche gli effetti di T_2 sulla base di dati in quanto il valore di Z prodotto da T_2 è calcolato a partire dal valore di X scritto da T_1 . Pertanto sia T_1 che T_2 devono essere rolled-back.

Per evitare questo fenomeno detto rollback a cascata occorre impedire alle transazioni di leggere dati sporchi. Ciò può essere ottenuto adottando un protocollo di locking a due fasi stretto. Una transazione soddisfa il protocollo a due fasi stretto se:

1. non scrive sulla base di dati fino a quando non ha raggiunto il suo punto di commit;
2. non rilascia un lock finché non ha finito di scrivere sulla base di dati.

La condizione 1 garantisce che se una transazione è abortita allora non ha modificato nessun item nella base di dati; la condizione 2 garantisce che quando una transazione legge un item scritto da un'altra transazione quest'ultima non può essere abortita (quindi nessuna transazione legge dati sporchi).

I protocolli di locking a due fasi stretti possono essere classificati in *protocolli conservativi* e *protocolli aggressivi*; i primi cercano di evitare il verificarsi di situazioni di stallo, i secondi cercano di processare le transazioni il più rapidamente possibile anche se ciò può portare a situazioni di stallo e, quindi, alla necessità di abortire qualche transazione.

La versione più conservativa di un protocollo di locking a due fasi stretto è quella che impone ad una transazione di richiedere all'inizio tutti gli item di cui può avere bisogno. Lo scheduler permette

alla transazione di procedere solo se tutti i lock che ha richiesto sono disponibili, altrimenti la mette in una coda di attesa. In tal modo non possono verificarsi deadlock, ma possono ancora verificarsi livelock. Per evitare anche il verificarsi di livelock si può impedire ad una transazione T di procedere (anche se tutti i lock da essa richiesti sono disponibili) se c'è un'altra transazione che precede T nella coda che è in attesa di ottenere un lock richiesto da T. E' facile vedere che se si adotta un protocollo in cui tutti i lock sono richiesti all'inizio e uno scheduler che garantisce ad una transazione T tutti i lock richiesti se e solo se:

1. tutti i lock sono disponibili;
2. nessuna transazione che precede T nella coda è in attesa di un lock richiesto da T.

Il protocollo conservativo esaminato presenta due inconvenienti. Infatti l'esecuzione di una transazione può essere ritardata dal fatto che non può ottenere un lock anche se molti passi della transazione potrebbero essere eseguiti senza quel lock; inoltre una transazione è costretta a richiedere un lock su ogni item che potrebbe essergli necessario anche se poi di fatto non l'utilizza.

La versione più aggressiva di un protocollo di locking a due fasi stretto è quella che impone ad una transazione di richiedere un lock su un item immediatamente prima di leggerlo o scriverlo. Se le transazioni soddisfano tale protocollo possono verificarsi situazioni di stallo; un modo per evitare ciò è quello di definire un ordinamento sugli item e di imporre alle transazioni di richiedere i lock in accordo a tale ordinamento.

Un protocollo aggressivo è adatto a situazioni in cui la probabilità che due transazioni richiedano un lock su uno stesso item è bassa (e quindi è bassa la probabilità che si verifichi una situazione di stallo), in quanto evita al sistema il sovraccarico dovuto alla gestione dei lock (decidere se garantire un lock su un dato item ad una data transazione, gestire la tavola dei lock, mettere le transazioni in una coda o prelevarle da essa). Un protocollo conservativo è invece adatto a situazioni opposte in quanto evita al sistema il sovraccarico dovuto alla gestione dei deadlock (rilevare e risolvere situazioni di stallo, eseguire parzialmente transazioni che poi vengono abortite, rilascio dei lock mantenuti da transazioni abortite).

7.6 Timestamp

I metodi per il controllo della concorrenza basati sui timestamp ordinano le transazioni in base ai loro timestamp. Il timestamp è assegnato ad una transazione dallo scheduler quando la transazione ha inizio.

In tale approccio al controllo della concorrenza, uno schedule è serializzabile se è equivalente allo schedule seriale in cui le transazioni compaiono ordinate in base al loro timestamp.

Dato uno schedule occorre verificare che, per ciascun item acceduto da più di una transazione, l'ordine con cui le transazioni accedono all'item non viola la serializzabilità dello schedule. A tale scopo vengono associati a ciascun item X due timestamp:

- il read timestamp di X, denotato con $read_TS(X)$, che è il più grande fra tutti i timestamp di transazioni che hanno letto con successo X;
- il write timestamp di X, denotato con $write_TS(X)$, che è il più grande fra tutti i timestamp di transazioni che hanno scritto con successo X.

Ogni volta che una transazione T cerca di eseguire un $read(X)$ o un $write(X)$, occorre confrontare il timestamp $TS(T)$ di T con il read timestamp e il write timestamp di X per assicurarsi che l'ordine basato sui timestamp non è violato.

Algoritmo 7.6.1 – Controllo della serializzazione

'algoritmo per il controllo della concorrenza opera nel modo seguente:

Ogni volta che una transazione T cerca di eseguire l'operazione $write(X)$:

1. se $read_TS(X) > TS(T)$, T viene rolled back;
2. se $write_TS(X) > TS(T)$, l'operazione di scrittura non viene effettuata;
3. se nessuna delle condizioni precedenti è soddisfatta allora l'operazione di scrittura è eseguita e $TS(T)$ diventa il nuovo valore di $write_TS(X)$.

Ogni volta che una transazione T cerca di eseguire l'operazione read(X):

1. se $\text{write_TS}(X) > \text{TS}(T)$, T viene rolled back;
2. se $\text{write_TS}(X) \leq \text{TS}(T)$, allora l'operazione di lettura è eseguita e se $\text{read_TS}(X) < \text{TS}(T)$, $\text{TS}(T)$ diventa il nuovo valore di $\text{read_TS}(X)$.

Consideriamo ad esempio le due transazioni seguenti con i timestamp: $\text{TS}(T_1)=110$, $\text{TS}(T_2)=100$ e consideriamo lo schedule di T_1, T_2 , notiamo che quando T_2 cerca di seguire (*) viene abortita:

T_1	T_2
read(X)	read(X)
$X:=X+10$	$X:=X+5$
write(X)	(*)write(X)

Consideriamo ora le due transazioni seguenti con i timestamp: $\text{TS}(T_1)=110$, $\text{TS}(T_2)=100$ e consideriamo lo schedule di T_1, T_2 , notiamo che in questo caso l'operazione (*) non viene eseguita:

T_1	T_2
read(Y)	read(Y)
$X:=Y+10$	$X:=Y+5$
write(X)	(*)write(X)